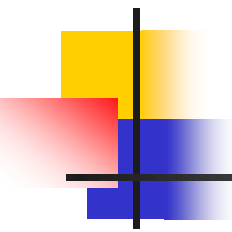


第2章 16位和32位微处理器

本章内容：

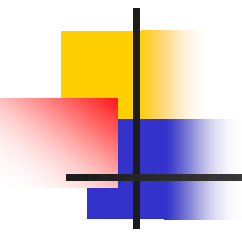
- 16位微处理器8086/8088
- 32位微处理器80386
- 32位微处理器Pentium



2.1 16位微处理器8086/8088

本节内容：

- 简介
- 8086/8088CPU的内部结构
- 8086/8088 CPU的寄存器结构
- 时钟周期、总线周期的概念
- 8086/8088CPU的引脚信号和功能
- 8086/8088系统的工作模式
- 8086/8088的操作时序
- 8086/8088的存储器组织



2.1.0 简介

- 8086：16位微处理器
 - 采用单一的+5V电源和40条引脚的双列直插式封装；时钟频率为5MHz~10MHz，最快的指令执行时间为0.4μs。
 - 8086有16根数据线和20根地址线，可以处理8位或16位数据，可寻址 2^{20} 即1MB的存储单元和64KB的I/O端口。
- 8088：准16位微处理器
 - 设计的主要目的是为了与Intel原有的8位外围接口芯片直接兼容。
 - 8088的内部寄存器、运算器以及内部数据总线都是按16位设计的，但外部数据总线只有8条，因此执行相同的程序，8088要比8086有较多的外部存取操作而执行得较慢。

2.1.1 8086/8088 CPU的内部结构

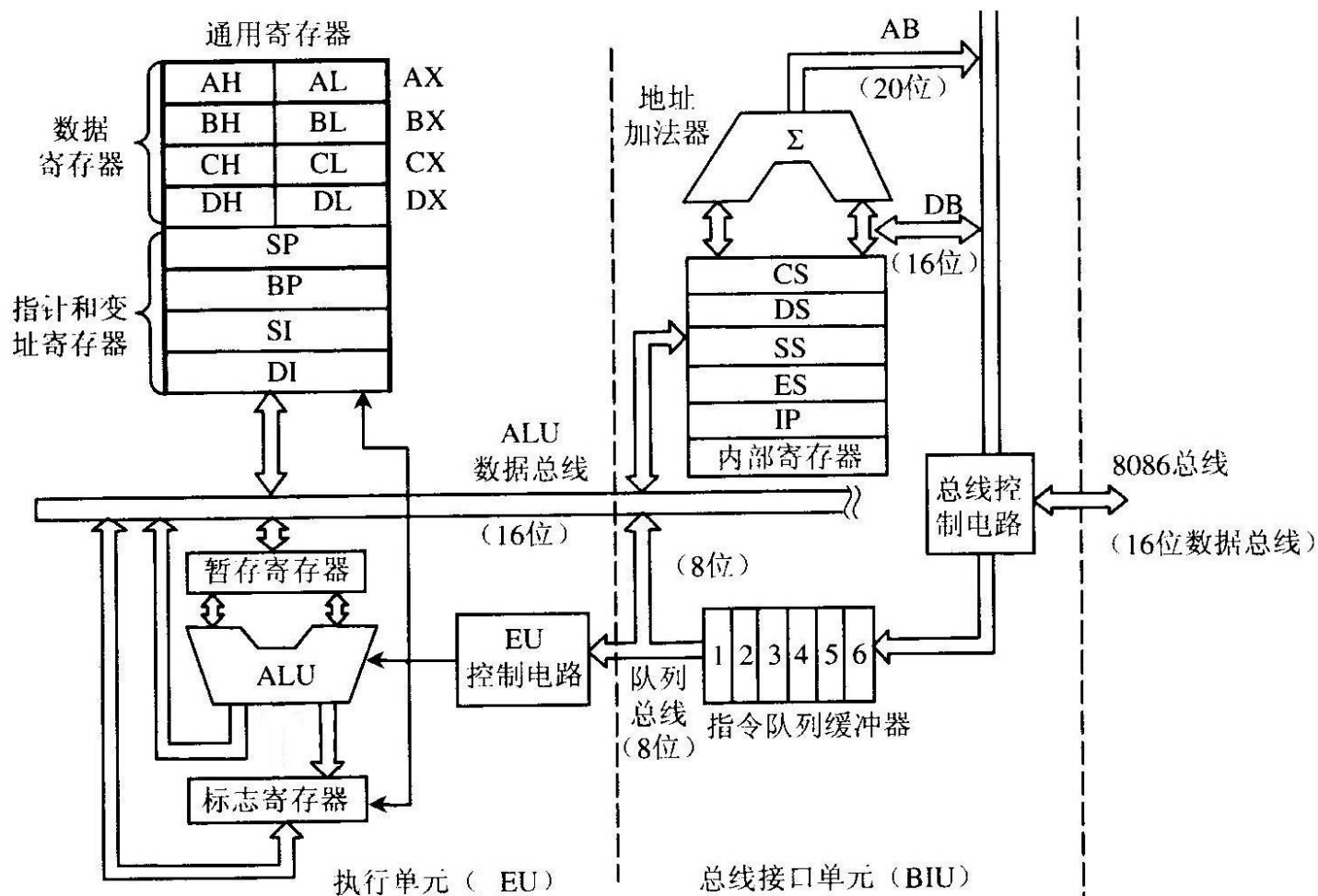
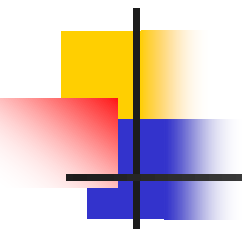


图2.1 8086微处理器内部结构框图

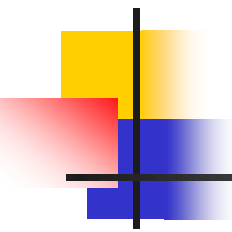


1.总线接口部件BIU

➤ 功能

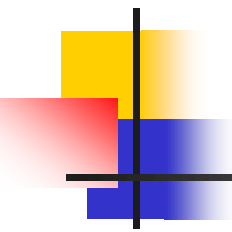
负责CPU与内存或I/O端口传送指令或数据。

- ❖ BIU从内存取指令送到指令队列缓冲器。
- ❖ 当EU执行指令时，BIU要配合EU从指定的内存单元或I/O端口中读取数据，或者把EU的操作结果送到指定的内存单元或I/O端口去。
- 组成：段寄存器、指令指针寄存器、地址加法器、指令预取队列及总线控制逻辑。



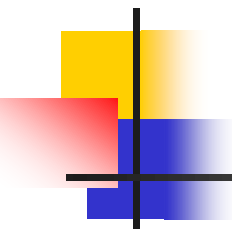
1)段寄存器

- 所有寄存器都是16位的，因此，能够提供的**最大地址空间只能为64 KB**。
- 为了寻址1MB，将存储器的空间分成若干段，**每段最大为64KB**。
- **段寄存器**：用来存放段的起始地址(16位)的寄存器，设有四个段寄存器：
 - **CS** 代码段寄存器(Code Segment register)
 - **DS** 数据段寄存器(Data Segment register)
 - **SS** 堆栈段寄存器(Stack Segment register)
 - **ES** 附加数据段寄存器(Extra Segment register)



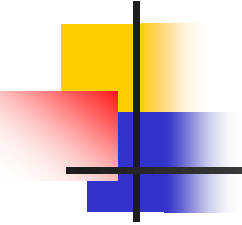
2)地址加法器

由于8086内部寄存器都是16位的，需要一个附加结构-地址加法器来根据提供的16位信息产生20位地址。



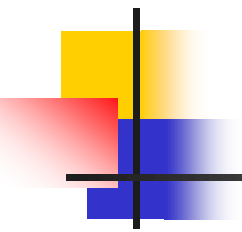
3)指令预取队列(指令队列缓冲器)

- 8086：指令预取队列为6字节
- 8088：指令预取队列为4字节
- 指令预取队列采用“**先进先出**”原则。
- 要执行的指令预先由BIU从内存取出放在队列中，然后EU再从队列中取出指令并执行。
- 一般情况下，EU每执行完一条指令，就可以立即从指令队列中取指令执行，从而提高了CPU的效率。



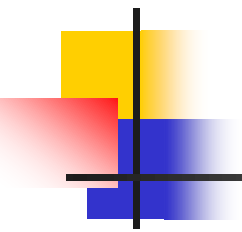
4)总线控制逻辑

- 8086分配20条引脚线传送20位地址、16位数据和4位状态信息，这就必须要分时传送。
- 总线控制逻辑的功能，就是以逻辑控制方法实现上述信息的分时传送。



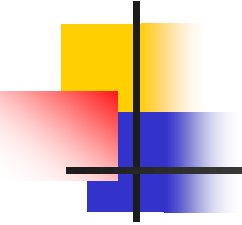
2.执行部件EU

- 功能
 - 负责指令的译码、执行和数据运算。
 - 负责向总线接口部件BIU提供偏移地址。
 - 对通用寄存器和标志寄存器进行管理。
- 组成：算术逻辑单元(ALU)、1个标志寄存器、8个通用寄存器、1个数据暂存寄存器和EU控制电路。



1)算术逻辑部件ALU

用于进行8位和16位的**算术和逻辑运算**，也可以按照指令的寻址方式**计算出寻址单元的16位偏移量**。

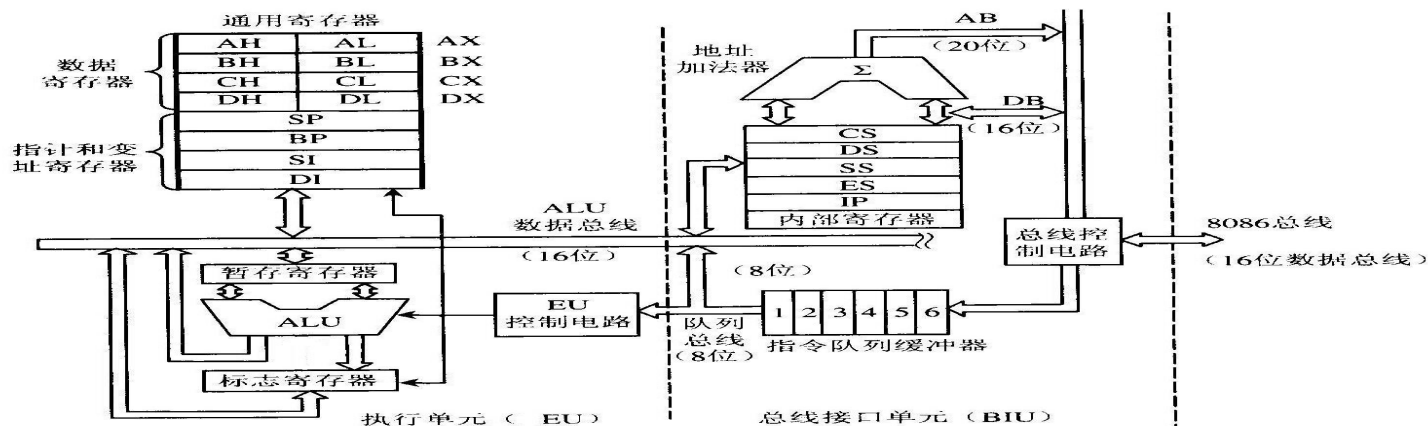


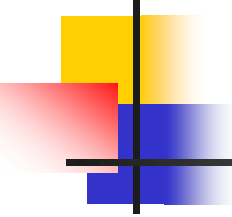
2)标志寄存器FR

16位，用来反映CPU运算的状态特征或存放控制标志。

3)通用寄存器组

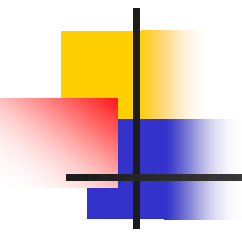
- 4个16位数据寄存器**AX**、**BX**、**CX**、**DX**
- 4个16位指针与变址寄存器：
堆栈指针寄存器**SP**(Stack Pointer)
基址指针寄存器**BP**(Base Pointer)
源变址寄存器**SI**(Source Index)
目的变址寄存器**DI**(Destination Index)





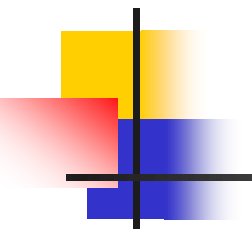
4)数据暂存寄存器

协助ALU完成运算，暂存参加运算的数据。



5)EU控制电路

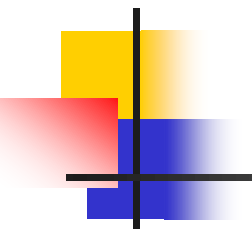
- 它是控制、定时与状态逻辑电路，接收从BIU中指令队列取来的指令，经过指令译码形成各种定时控制信号，对EU的各个部件实现特定的定时操作。
- EU中所有的寄存器和数据通道(除指令队列总线为8位外)都是16位的宽度，可实现数据的快速传送。



3.BIU和EU的流水线管理

(1)每当8086的指令队列中有**2个空字节**或8088的指令队列有**1个空字节**，BIU就会自动把后面的指令从存储器取到指令队列中，从而提高了CPU执行指令的速度。

(2)每当EU准备执行一条指令时，它会从指令队列前部取出指令，进行译码，然后去执行。在执行指令时，如果必须访问存储器或I/O端口，EU就会请求BIU去完成访问外部的操作，如果此时BIU正好处于空闲状态，那么会立即响应EU的请求。若EU向BIU发出请求访问时，BIU正在将某条指令取到指令队列中，此时BIU首先完成取指令操作，然后再去响应EU发出的访问外界的请求。



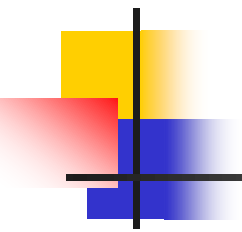
3.BIU和EU的流水线管理(续)

- (3)当指令队列已满，且EU对BIU又没有总线访问请求时，BIU进入**空闲状态**。
- (4)**在执行转移指令、调用指令和返回指令时**，如果要执行的指令不在指令队列中，则指令队列中原有内容被自动清除，BIU会重新取指令，把将要转入的程序段的指令装入到指令队列中。

2.1.2 8086/8088 CPU的寄存器结构

AH	AL	累加器	}	数据寄存器	}	通用寄存器					
BH	BL	基址寄存器									
CH	CL	计数寄存器									
DH	DL	数据寄存器									
SP		堆栈指针寄存器	}	地址指针和 变址寄存器							
BP		基址指针寄存器									
SI		源变址寄存器									
DI		目的变址寄存器									
IP		指令指针寄存器									
FR		标志寄存器									
CS		代码段寄存器	}	段寄存器							
DS		数据段寄存器									
SS		堆栈段寄存器									
ES		附加段寄存器									

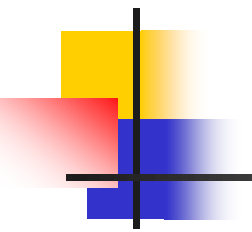
图 2.2 8086/8088 的寄存器结构



1.通用寄存器

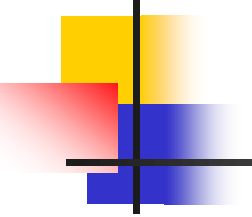
分为：

- 数据寄存器
- 地址寄存器



1)数据寄存器

- EU中有4个16位的数据寄存器AX、BX、CX、DX
- 每个数据寄存器又可分为**高字节H**和**低字节L**寄存器，即AH、BH、CH、DH和AL、BL、CL、DL两组。
- **16位数据寄存器主要用于存放数据，也可存放地址，而8位寄存器只能用于存放数据，它们均可以用寄存器名来独立寻址、独立使用。**



2)地址指针寄存器和变址寄存器

- 都是16位，**一般用来存放偏移地址。**
- **指针寄存器SP和BP用来存取位于当前堆栈段中的数据**，但SP和BP使用上有区别。
- **堆栈指针寄存器SP**给出栈顶的偏移地址。
- **基址指针寄存器BP**用来存放位于堆栈段中的一个数据区基址的偏移地址。
- **源变址寄存器SI和目的变址寄存器DI**用来存放当前数据段的偏移地址。

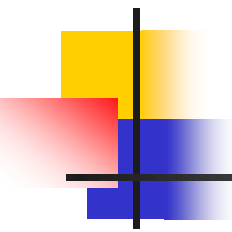
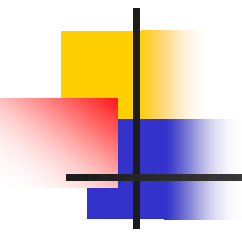


表2.1 寄存器的隐含使用

表 2.1 数据寄存器的隐含使用

寄存器	执 行 操 作
AX	整字乘法、整字除法和整字 I/O
AL	字节乘法、字节除法、字节 I/O、查表和十进制算术运算
AH	字节乘法和字节除法
BX	查表
CX	字符串操作和循环
CL	变量的移位和循环移位
DX	整字乘法、整字除法和间接寻址 I/O
SP	堆栈操作
SI	字符串操作
DI	字符串操作

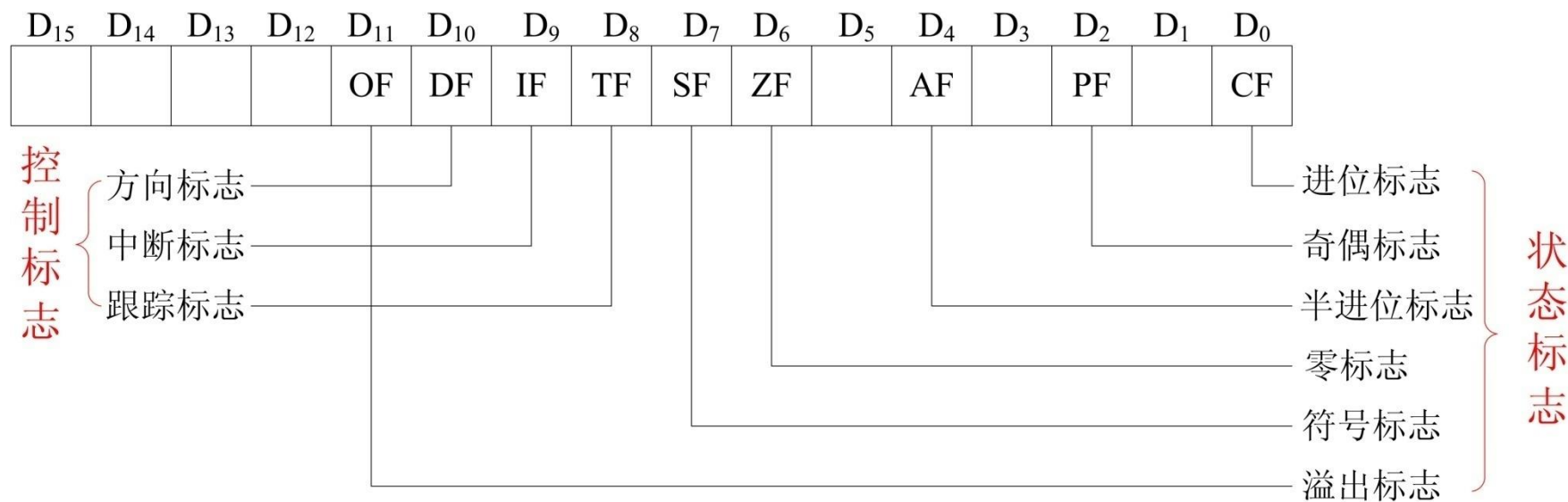


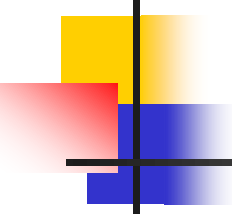
2.指令指针寄存器

- 16位，存放着BIU要取的下一条指令的偏移地址。
- 指令执行时，每取一次指令IP就自动加1，这样保证能按顺序取出并执行指令。
- 指令代码存放在存储器的代码段，CPU利用CS和IP取得要执行的指令。
- 修改IP中的内容，就可以改变指令的执行流向。

3.标志寄存器(16位)

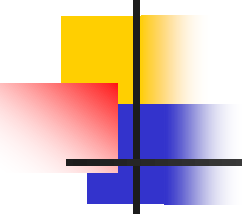
- 16位标志寄存器FR用于反映指令执行结果或控制指令执行的形式。
- 只用了其中的9位，分为：状态标志位和控制标志位。





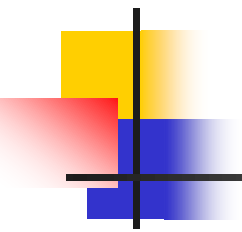
1)状态标志位(6位)

用来反映算术或逻辑运算后结果的状态，以记录CPU的状态特征。



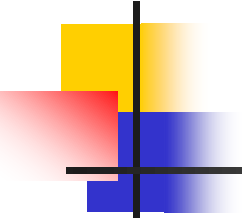
进位标志CF(Carry Flag)

- 加法时，最高位（字节操作时的 D_7 位，字操作时的 D_{15} 位）是否有**进位**产生。
- 减法时，最高位（字节操作时的 D_7 位，字操作时的 D_{15} 位）是否有**借位**产生。



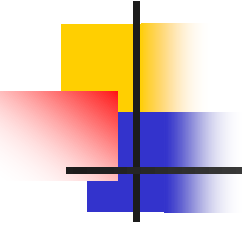
奇偶标志PF(Parity Flag)

- 若运算结果低8位中“1”的个数为偶数，则PF=1；否则PF=0。
- 一般用来检测数据传输中是否发生错误。



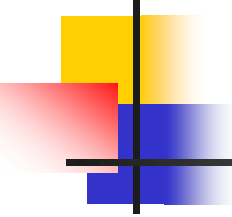
辅助进位标志AF(Auxiliary carry Flag)

- 加法时，第3位向第4位有进位。
- 减法时，第3位向第4位有借位。
- 该标志位通常用于对BCD算术运算结果进行调整。



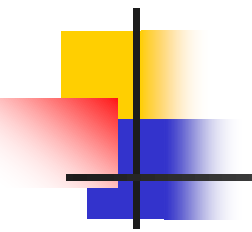
零标志ZF(Zero Flag)

若运算结果为0，则 $ZF=1$ ；否则 $ZF=0$ 。



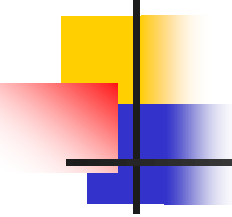
符号标志SF(Sign Flag)

它和运算结果的最高位 D_{15} 或 D_7 相同。



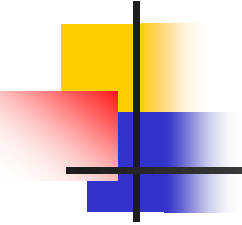
溢出标志OF(Overflow Flag)

若运算过程中发生了“溢出”，则OF=1，否则OF=0。字节运算大于127或小于-128时，标志位置1，当字运算大于32767或小于-32768，标志位置1。



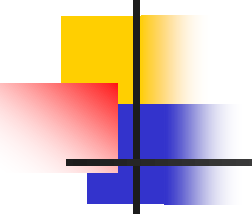
2)控制标志位(3位)

用来控制CPU的操作， 由程序设置或清除。



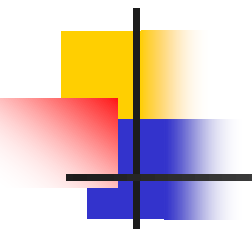
中断允许标志IF(Interrupt Enable Flag)

- 如果IF置“1”，则CPU可以接受**可屏蔽中断**请求；反之，则CPU不能接受可屏蔽中断请求。
- STI 使IF置“1”，即开放中断。
CLI 使IF清“0”，即关闭中断。



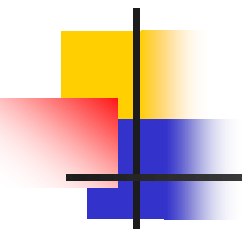
方向标志DF(Direction Flag)

- 控制字符串操作指令的步进方向。
DF=1时，地址自动递减；
DF=0时，地址自动递增。
- STD使DF=1。
CLD使DF=0。



跟踪(陷阱)标志TF(Trap Flag)

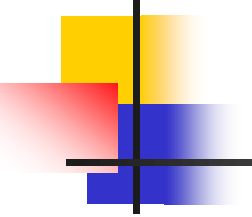
- ▶ 为调试程序的方便而设置的。
TF=1, 则处于单步工作方式;
TF=0, 将正常执行程序。
- ▶ 例如, 在系统调试软件DEBUG中的T命令, 就是用该标志位来进行程序的单步跟踪的。



4.段寄存器

介绍如下内容：

- 存储器分段的概念
- 逻辑地址和物理地址
- 堆栈
- 段寄存器的使用



1)存储器分段的概念

- 8086/8088有20位地址线，能够寻址1 MB的内存空间；
- CPU内部存放地址信息的IP、SP、SI、DI或BX等寄存器却只有16位，只能寻址64KB存储空间。
- 所谓分段技术就是把1MB的存储空间分成若干逻辑段，每个逻辑段最大具有64 KB的存储空间。
- 段内地址是连续的，段与段之间是相互独立的。逻辑段可以在整个存储空间浮动，即段的排列可以连续、分开、部分重叠或完全重叠，非常灵活。

图2.4 逻辑分段的示意图

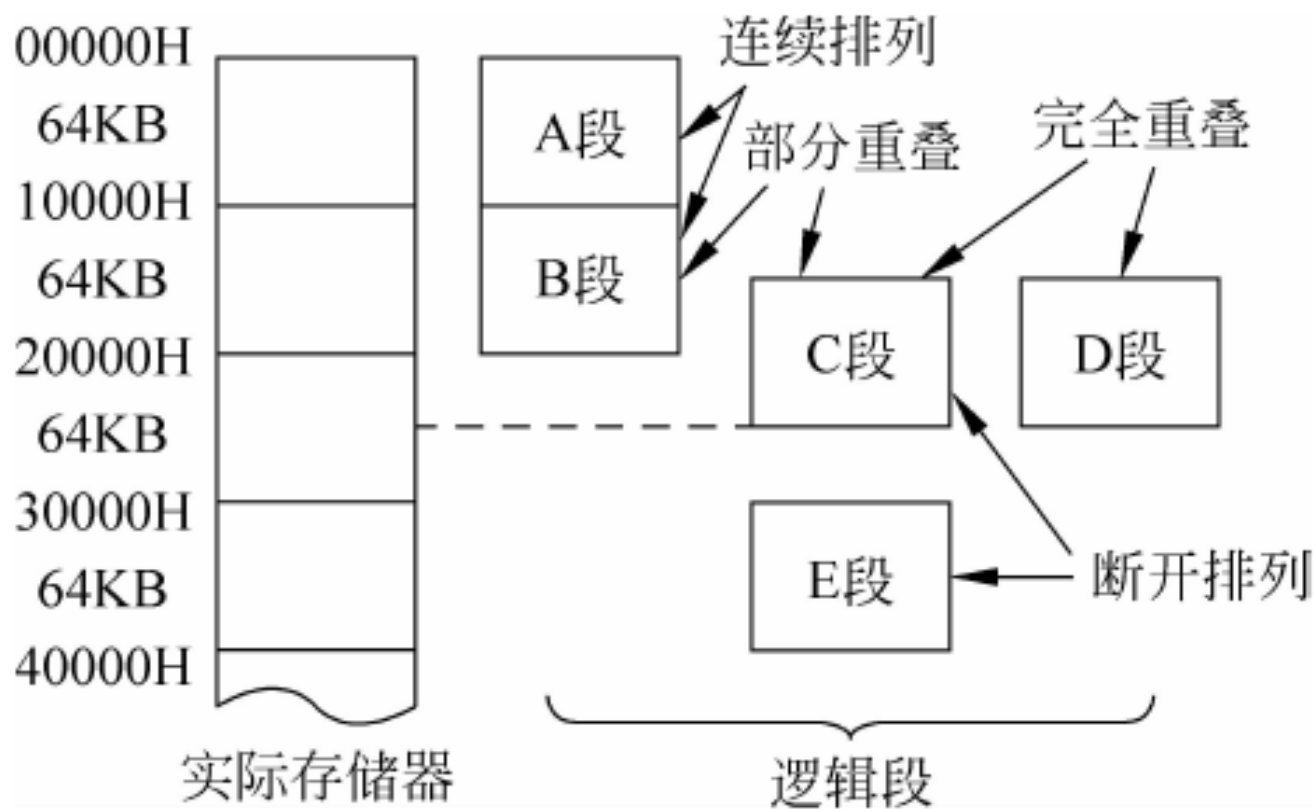
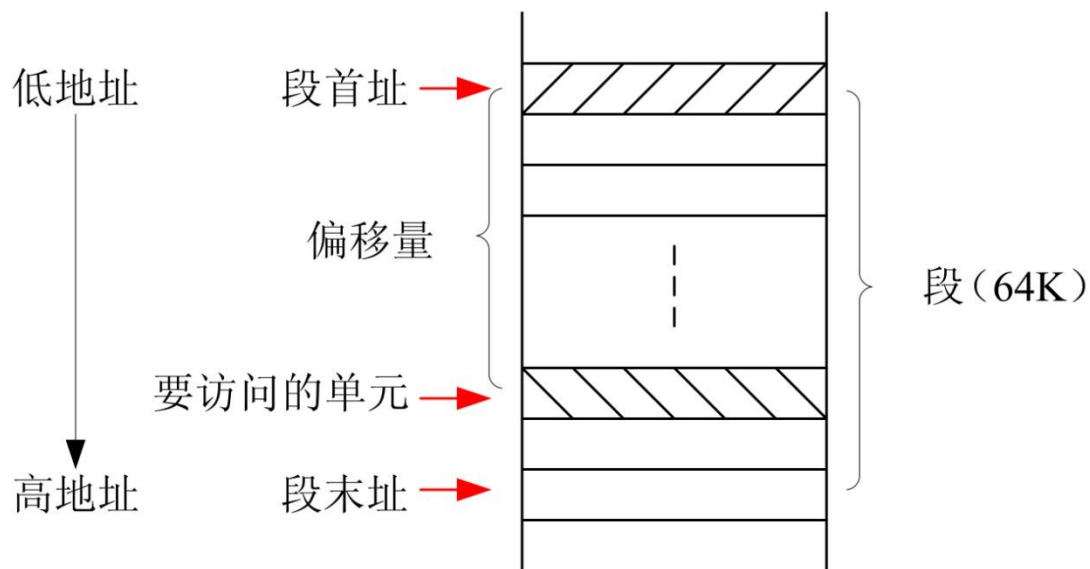


图 2.4 逻辑分段的示意图

2)逻辑地址和物理地址

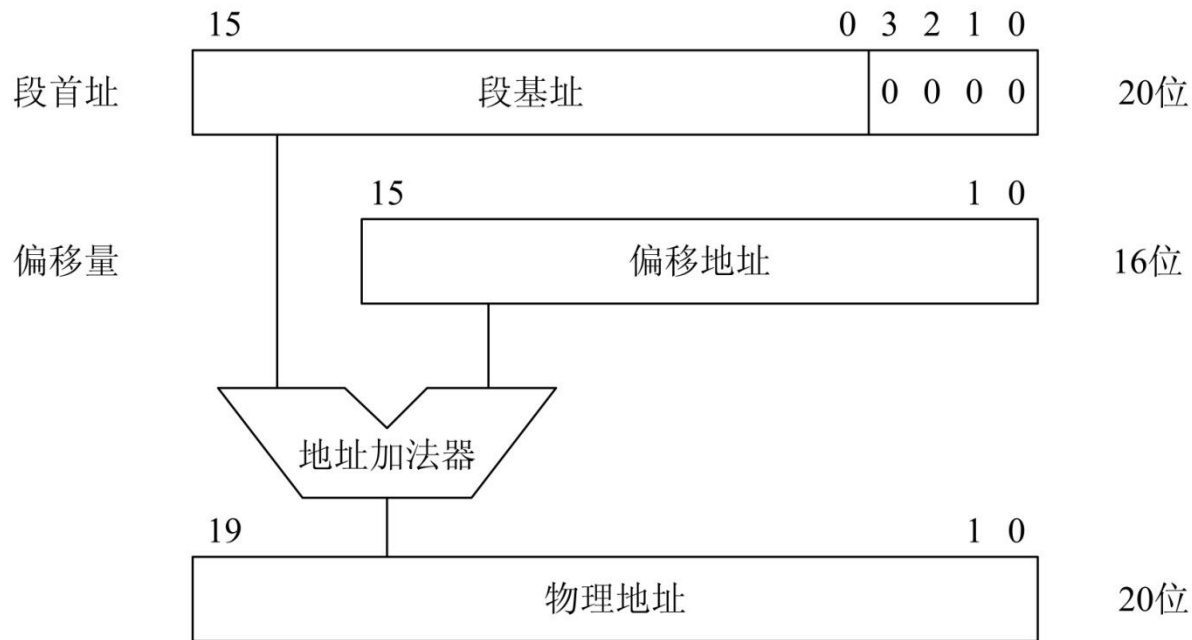
- **段首址**：段的第一个单元的地址(20位)，最低4位是全0(即段首址是16的整数倍)。
- **段基址**：段首址的高16位。段基址存放在段寄存器中。
- **偏移地址**：段内存储单元距离段首址的偏移量(以字节数计算，16位)，也称有效地址EA。偏移地址存放在IP、BP、SI、DI或BX中，或者是通过计算得到。
- **逻辑地址**：通常用**段基址：偏移地址**的形式来描述，在程序中使用。



物理地址的形成

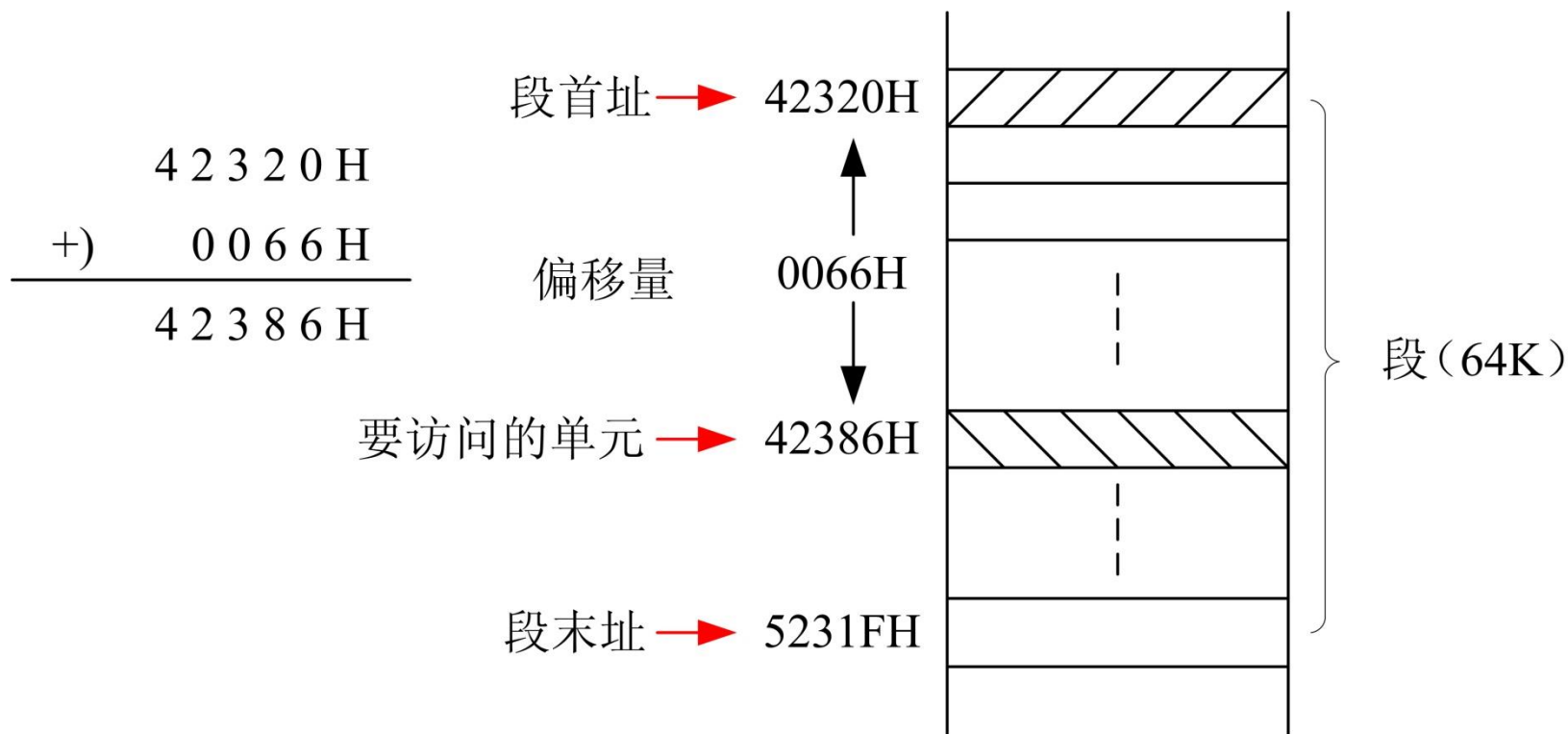
- **物理地址**：指CPU和存储器进行数据交换时实际寻址所使用的地址，是用20位二进制数或5位十六进制数表示的地址。
- 任何一个单元的20位物理地址都是由它的逻辑地址变换得到的：

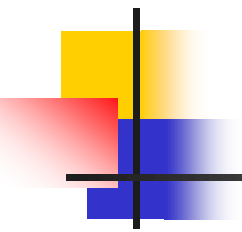
$$\text{物理地址} = \text{段基址} \times 16 + \text{偏移地址}$$



例题

设(CS)=4232H , (IP)=0066H, 试计算物理地址。

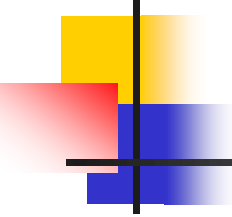




注意

一个存储单元的物理地址是唯一的，而逻辑地址是可以不唯一的。

- 例如，物理地址是12345H，它的逻辑地址可以是1233H：0015H
- 也可以是1234H：0005H。



3)堆栈

- 堆栈是以“**先进后出**”或“**后进先出**”原则管理的存储区域。
- 堆栈段也采用段定义语句在存储器中定义，最大为64KB。可在1MB的存储空间内浮动。
- 堆栈段所在存储区中的位置由堆栈段寄存器SS和堆栈指针SP来指示。
- **SS给出堆栈段的段基址**
- **SP存放栈顶地址**，指出从栈顶到段首址的偏移量。
- 栈顶与栈底之间单元中的内容是堆栈段中的有效数据。
- 堆栈操作有**入栈(PUSH)**和**出栈(POP)**两种。

SS=1050H, SP=008H, AX=1234H
 PSHH AX
 POP AX
 POP BX

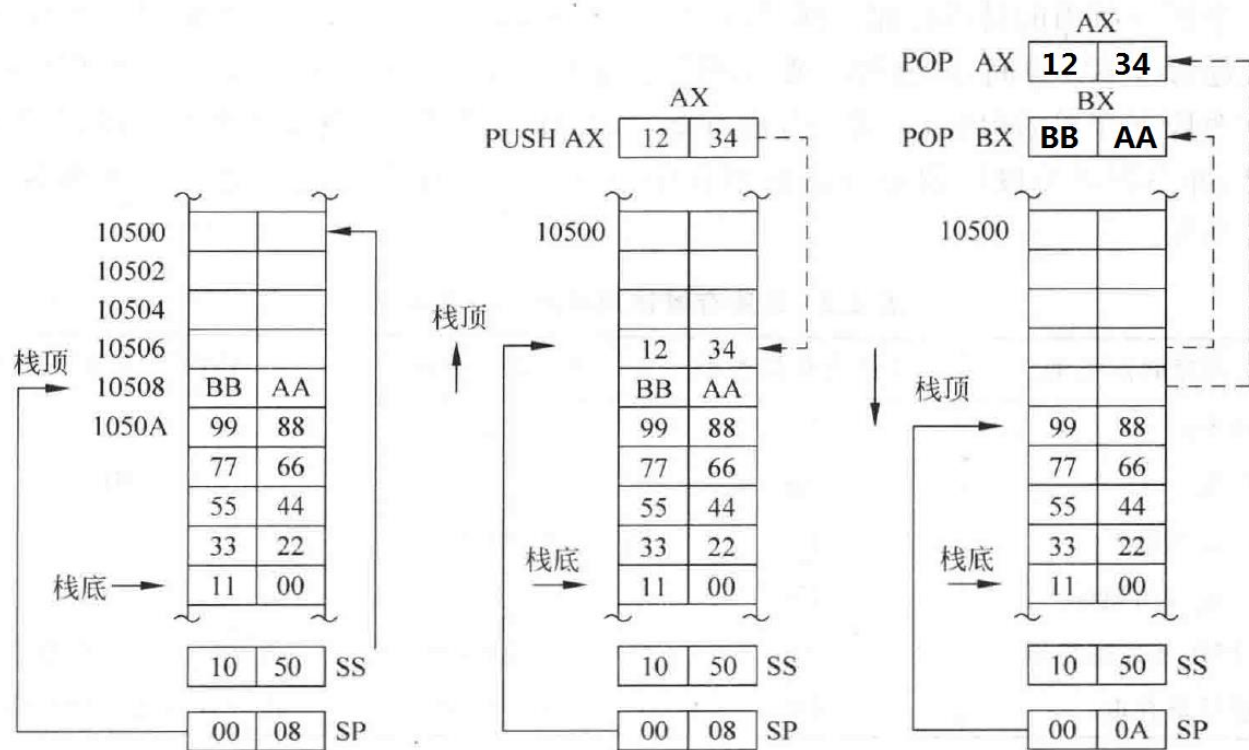
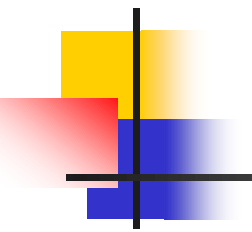


图 2.6 8086 系统堆栈及其入栈出栈操作



4)段寄存器的使用

四个段寄存器分别指明四个现行可寻址的逻辑段：

- (1) **代码段(Code Segment)**：用来存放当前正在运行的程序。系统在取指令时将寻址代码段，其段基址和偏移地址分别由段寄存器CS和指令指针IP给出。
- (2) **数据段(Data Segment)**：存放当前运行程序所用的数据。用户在寻址该段内的数据时，可以缺省段的说明，其偏移地址可通过多种寻址方式形成。
- (3) **堆栈段(Stack Segment)**：堆栈为保护、调度数据提供了重要的手段。系统在执行栈操作指令时将寻址堆栈段，段基址和偏移地址分别由段寄存器SS和堆栈指针SP提供。
- (4) **附加数据段(Extra Segment)**：该段是一个辅助的数据区，也用于数据的保存。用户在访问段内的数据时，其偏移地址同样可以通过多种寻址方式来形成，但在偏移地址前要加上段的说明(即段跨越前缀ES)。

图2.7 段寄存器的使用情况

只要修改段寄存器的内容，就可以将相应的存放区设置在内存存储空间的任何位置上。这些区域可以相互独立，也可以部分或完全重叠。

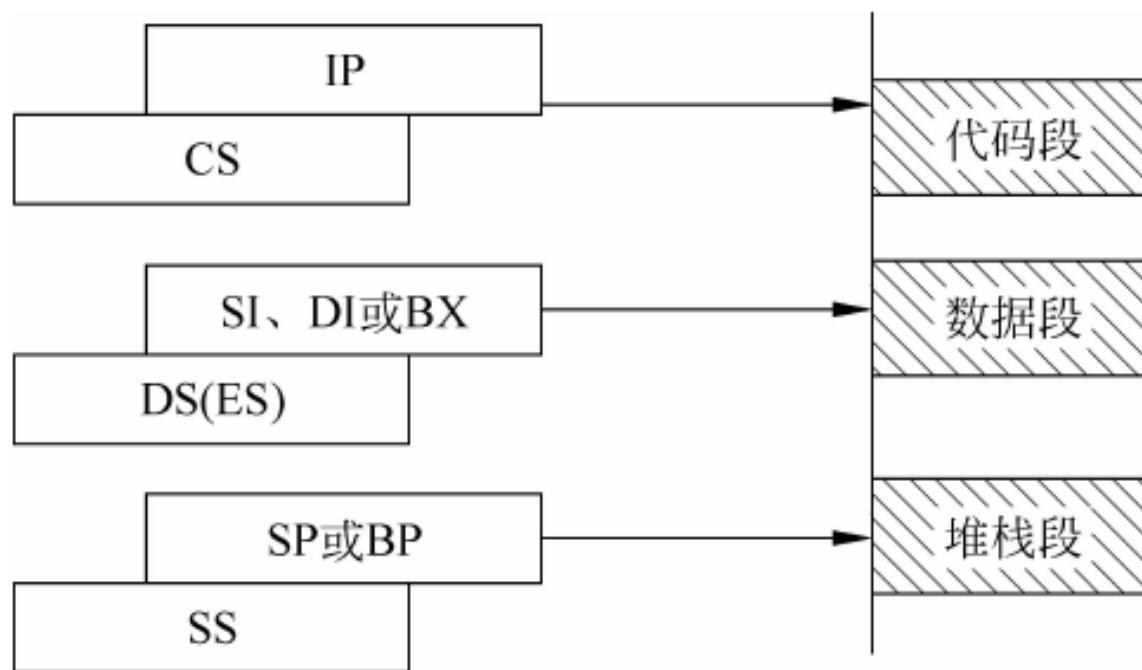


图 2.7 段寄存器的使用情况

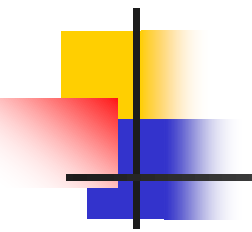
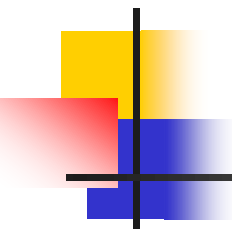


表2.2 段寄存器使用时的一些基本约定

表 2.2 段寄存器使用时的一些基本约定

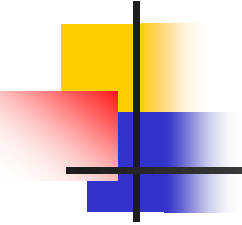
访问存储器类型	默认寄存器类型	可指定段寄存器	段内偏移地址来源
取指令码	CS	无	IP
堆栈操作	SS	无	SP
串操作源地址	DS	CS、ES 和 SS	SI
串操作目的地址	ES	无	DI
BP 用作基址寄存器	SS	CS、DS 和 ES	按寻址方式求得有效地址
一般数据存取	DS	CS、ES 和 SS	按寻址方式求得有效地址



2.1.3 时钟周期、总线周期的概念

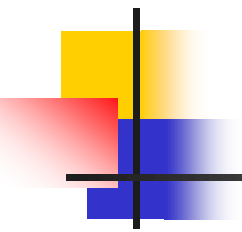
本小节内容：

- 时钟周期
- 总线周期与指令周期



1.时钟周期

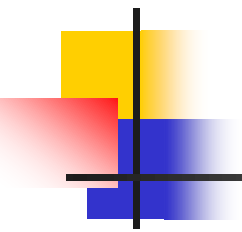
- 时钟周期是CPU的基本时间计量单位，是CPU工作的最小时间单位，也称节拍脉冲或T周期，由主频决定。
- 对于8086来讲，若其主频为5MHz，则一个时钟周期为200ns。



2.总线周期与指令周期

总线周期

- 8086CPU通过总线对存储器或I/O端口进行一次信息的输入或输出过程，称为**总线操作**，执行该操作所需要的时间，称为**总线周期**。
- 在8086中，一个最基本的总线周期由4个时钟周期组成，每个时钟周期称为T状态，因此基本总线周期用 T_1 、 T_2 、 T_3 、 T_4 表示。



2.总线周期与指令周期(续)

指令周期

- 每条指令的执行由取指令、分析指令和执行指令等操作完成，执行一条指令所需要的时间称为一个指令周期。
- 简单指令执行时间就比较短，而复杂指令执行的时间就比较长，因此，不同指令的指令周期是不相同的。
- 一个指令周期由一个或几个总线周期组成，一个总线周期又由若干个时钟周期组成。

图2.8 典型的8086总线周期序列

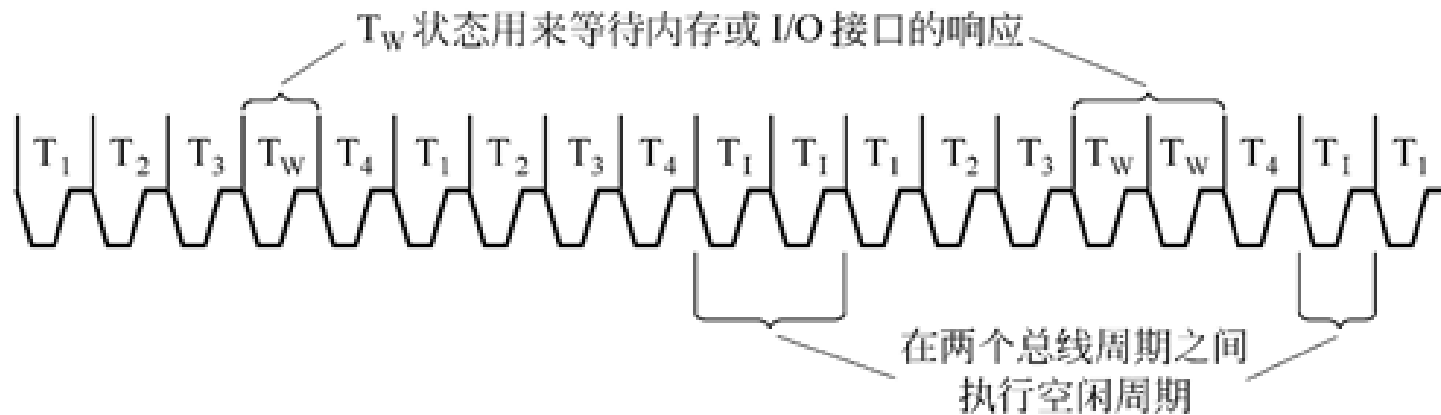
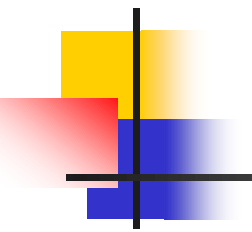


图 2.8 典型的 8086 总线周期序列



2.1.4 8086/8088 CPU的引脚信号和功能

本小节内容：

- 8086最小模式下引脚的功能定义
- 8086最大模式下引脚的功能定义
- 8088的引脚特性

图2.9 8086/8088CPU的引脚信号

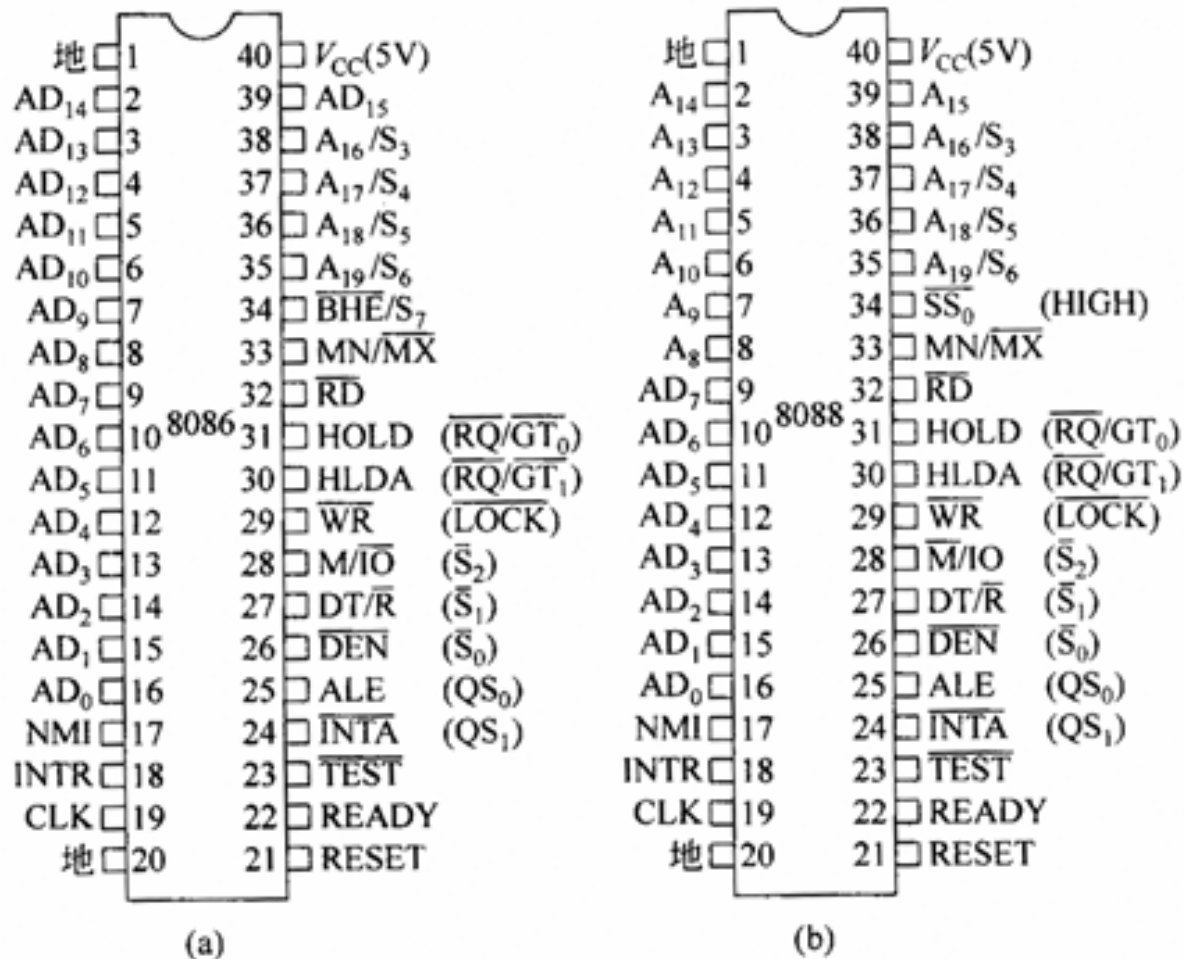
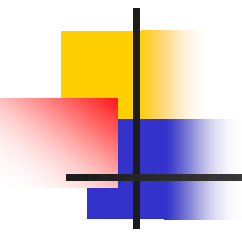


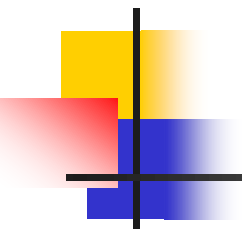
图 2.9 8086/8088 的引脚信号(括号中为最大模式时引脚名)



1.8086最小模式下引脚的功能定义

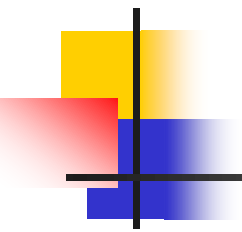
$\overline{\text{MN}}/\overline{\text{MX}}$

- 最小/最大模式设定，输入。
- $\overline{\text{MN}}/\overline{\text{MX}}=1$ 时：工作方式设置为**最小模式**，在此方式下，系统全部控制信号由8086本身提供；
 $\overline{\text{MN}}/\overline{\text{MX}}=0$ 时：工作方式设置为**最大模式**。



$AD_{15} \sim AD_0$

- 地址/数据**复用**引脚。
- 在总线周期的 T_1 状态用来输出要访问的存储器或I/O端口地址，在 $T_2 \sim T_4$ 状态，作为数据传输线。
- 传送地址时为三态**输出**，传送数据时可**双向**三态输入/输出。
- 在8088中，只有 $AD_7 \sim AD_0$ 8条地址/数据线， $AD_{15} \sim AD_8$ 只用来输出地址。



$A_{19}/S_6 \sim A_{16}/S_3$

- 地址/状态**复用**引脚，三态，分时输出。
- 在 T_1 状态：输出高4位地址。
- **访问存储器时**： $A_{19} \sim A_{16}$ 与 $AD_{15} \sim AD_0$ 组成20位地址；
- **访问I/O端口时**：**不使用这4条引线。**
- 在 $T_2 \sim T_4$ 状态：输出状态信息。

$S_6=0$ ，表示当前8086/8088与总线相连(即 S_6 始终保持低电平)。

S_5 用来指示状态寄存器中的中断允许标志IF的状态 (S_5 为1允许中断， S_5 为0禁止中断)。

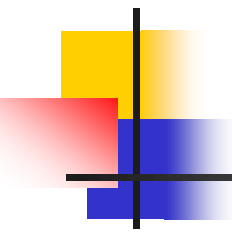


表2.3 S_4 和 S_3 的组合

表 2.3 S_4 和 S_3 代码指示正在使用的段寄存器

S_4	S_3	状 态
0	0	当前正在使用 ES
0	1	当前正在使用 SS
1	0	当前正在使用 CS(或未用任何段寄存器)
1	1	当前正在使用 DS

$\overline{\text{BHE}}/\text{S}_7$

- 高8位数据总线允许/状态复用引脚，三态，输出。
- 在 T_1 状态： $\overline{\text{BHE}}$ 输出，若 $\overline{\text{BHE}}=0$ ，表示高8位数据线 $D_{15} \sim D_8$ 上的数据有效。
- 在 $T_2 \sim T_4$ 状态：输出 S_7 (无意义)。

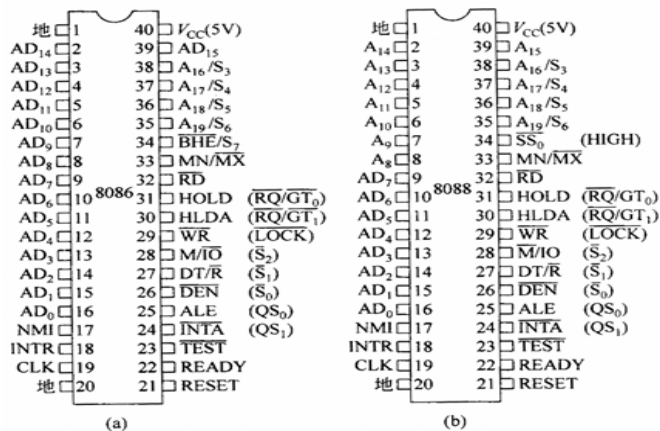
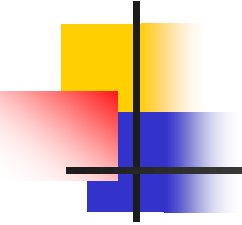
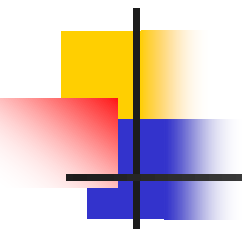


图 2.9 8086/8088 的引脚信号 (括号中为最大模式时引脚名)



ALE

- 地址锁存允许信号，输出，**正脉冲**。
- 提供给地址锁存器8282的控制信号。
- 在任何一个总线周期的**T₁状态**，ALE输出有效电平，以表示当前总线上输出的是地址信息，要求进行地址锁存。

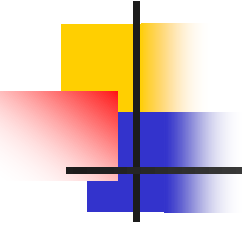


RESET

- 复位信号，输入，高电平有效。
- **至少维持4个时钟周期的高电平**，接通电源时间不能小于 $50\mu\text{s}$ 。

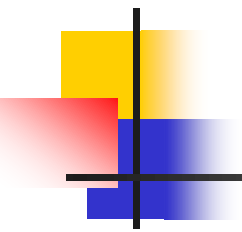
表 2.4 复位后 CPU 内部寄存器的状态

内部寄存器	状态	内部寄存器	状态
状态寄存器	清除	SS	0000H
IP	0000H	ES	0000H
CS	FFFFH	指令队列	清除
DS	0000H		



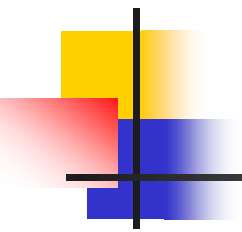
READY

- “准备好” 信号，输入，高电平有效
- 由所寻址的存储器或I/O端口发来的**响应信号**，表明存储器或I/O端口的状态。
- CPU在 T_3 采样READY，若READY=0，则在 T_3 后插入一个或多个 T_w ，直至READY=1，进入 T_4 ，完成数据传送，从而结束当前总线周期。



$\overline{RD}$

- 读信号，三态、输出。
- $\overline{RD} = 0$ 时，表示将要执行一个对存储器或I/O端口的读操作。



$\overline{\text{WR}}$

- 写信号，三态、输出。
- $\overline{\text{WR}} = 0$ 时，表示将要执行一个对存储器或I/O端口的写操作。

M/ $\overline{\text{IO}}$

- 存储器/外设控制信号，输出
 $\text{M}/\overline{\text{IO}} = 1$ ：访问存储器
 $\text{M}/\overline{\text{IO}} = 0$ ：访问I/O端口
- 一般被用于存储芯片或I/O接口芯片的片选($\overline{\text{CS}}$)译码电路中。

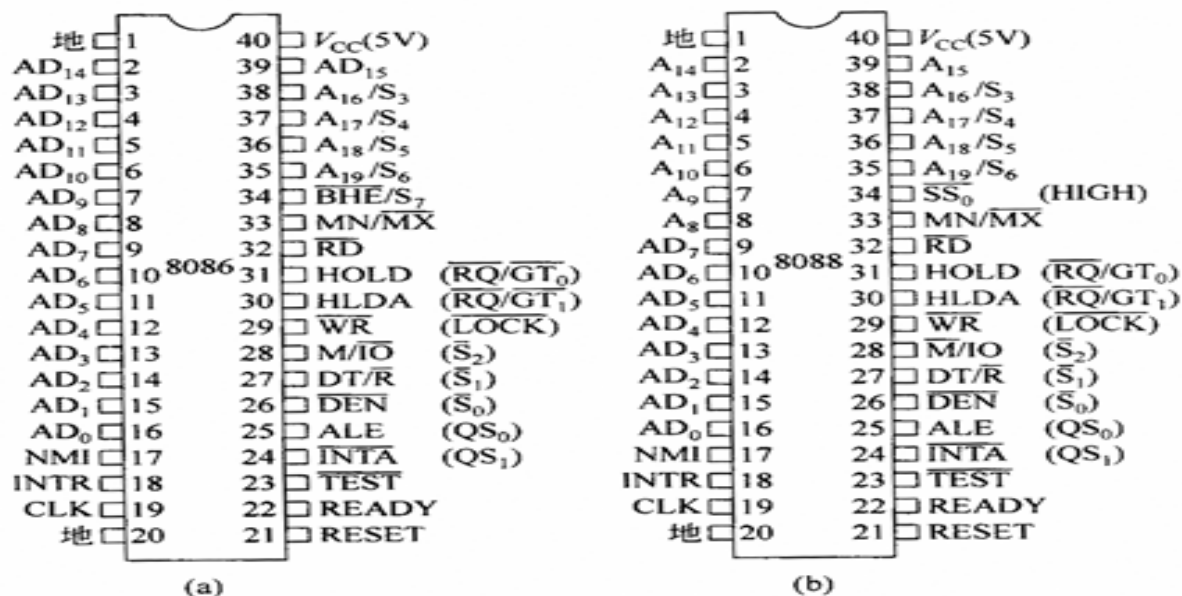
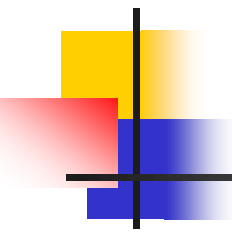
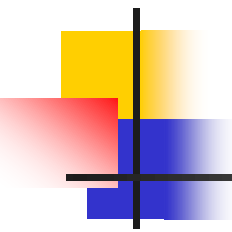


图 2.9 8086/8088 的引脚信号(括号中为最大模式时引脚名)



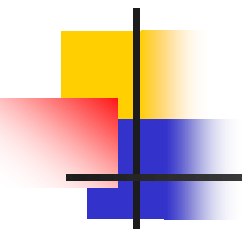
NMI

- 非屏蔽中断请求信号，输入，**上升沿有效**。
- 此请求**不受IF的影响**。只要此信号一出现，CPU就在现行指令结束后响应中断。



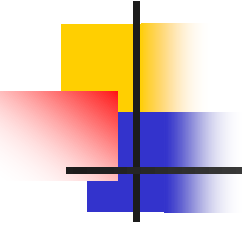
INTR

- 可屏蔽中断请求信号，输入，高电平有效。
- 当INTR=1时，表示外设提出了中断请求，8086/8088在每个指令周期的最后一个T状态去采样此信号。
- 若IF=1，则CPU响应中断，转去执行中断服务程序。



$\overline{\text{INTA}}$

- 中断响应信号，输出，低电平有效。
- 用于对外设的中断请求作出响应。是两个连续的**负脉冲**。
- 第1个脉冲是通知外设接口，它的中断请求已获允许；
- 第2个负脉冲要求外设接口往数据总线上发中断类型码，当外设接口收到第2个负脉冲后，往数据总线上放**中断类型码**。



HOLD、HLDA

- HOLD: 总线保持请求信号, 输入, 高电平有效
HLDA: 总线保持响应信号, 输出, 高电平有效
- HOLD信号是系统中的其它总线主控部件向CPU发出的**请求占用总线**的控制信号。当CPU从HOLD线上收到一个高电平请求信号时, 如果CPU允许让出总线, 就在当前总线周期完成时, 于T₄状态从HLDA线上发出一个**应答信号**, 对HOLD请求作出响应。同时, CPU使地址/数据总线和控制总线处于浮空状态, 从而让出了总线。
- 当请求部件完成对总线占用后, CPU就立即使HLDA变低, 同时恢复对总线的控制。

DT/ $\overline{R}$

- 数据收发控制信号，输出，三态。
- 用来控制数据总线驱动器8286的数据传送方向。
=1：发送数据
=0：接收数据

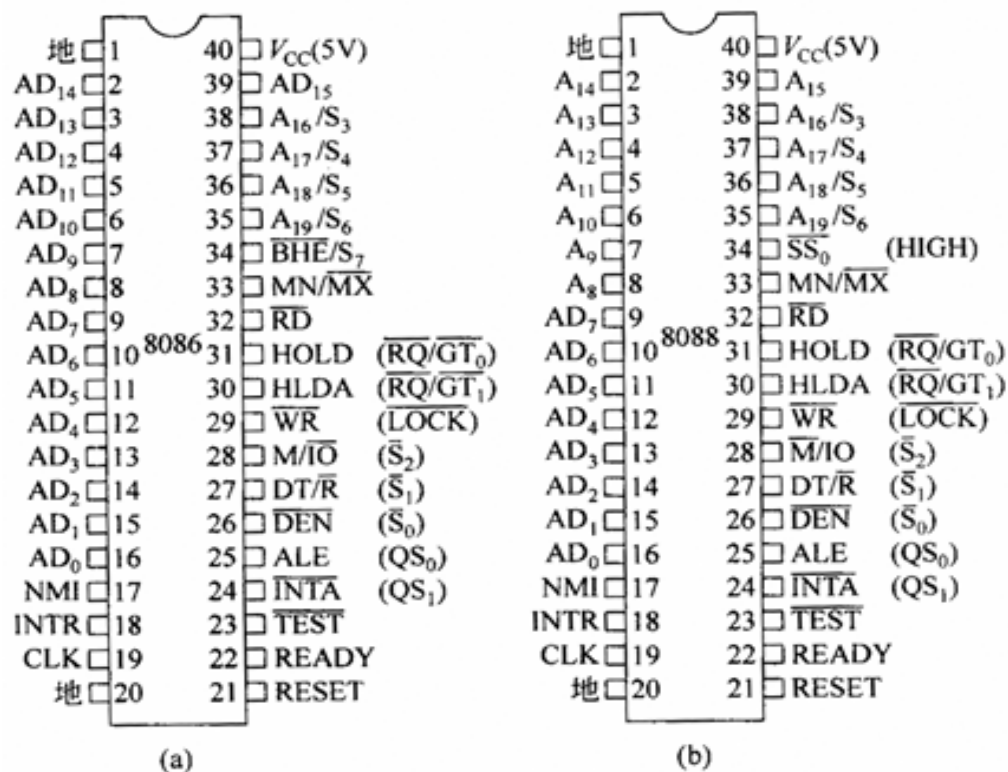
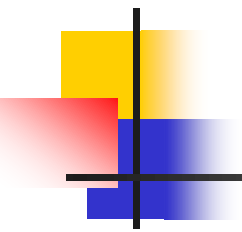
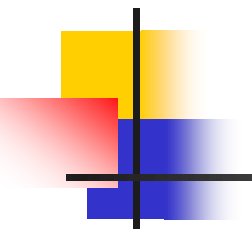


图 2.9 8086/8088 的引脚信号(括号中为最大模式时引脚名)



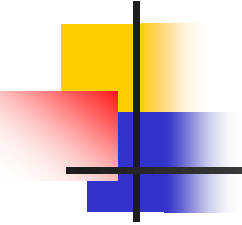
$\overline{\text{DEN}}$

- 数据允许信号，输出，低电平有效，三态。
- 提供给数据总线收发器8286，表示CPU准备发送或接收一个数据。
- 是8086提供给8286/8287（数据收发器）的选通信号，接至数据收发器的 $\overline{\text{OE}}$ 端。



$\overline{\text{TEST}}$

- 等待测试信号，输入，低电平有效。
- 用于多处理器系统中且只有在执行WAIT指令时才使用。
- 当CPU执行WAIT指令时，每隔5个时钟周期对该引脚进行一次测试：若 $\overline{\text{TEST}} = 1$ 时，CPU将停止取下条指令而进入等待状态，重复执行WAIT指令，直至该信号为0，CPU才继续往下执行被暂停的指令。
- 等待期间允许外部中断。
- WAIT指令可使CPU与外部硬件同步， $\overline{\text{TEST}}$ 相当于外部硬件的同步信号。



CLK

- 系统时钟，输入
- 通常与8284A时钟发生器的时钟输出端CLK相连。该时钟信号的低/高之比常采用2：1。
- 8086 CPU的标准时钟频率为5 MHz。

V_{CC} 、GND

- 电源线 V_{CC} 接入的电压为 $+5V \pm 10\%$ 。
- 两条地线GND均应接地。

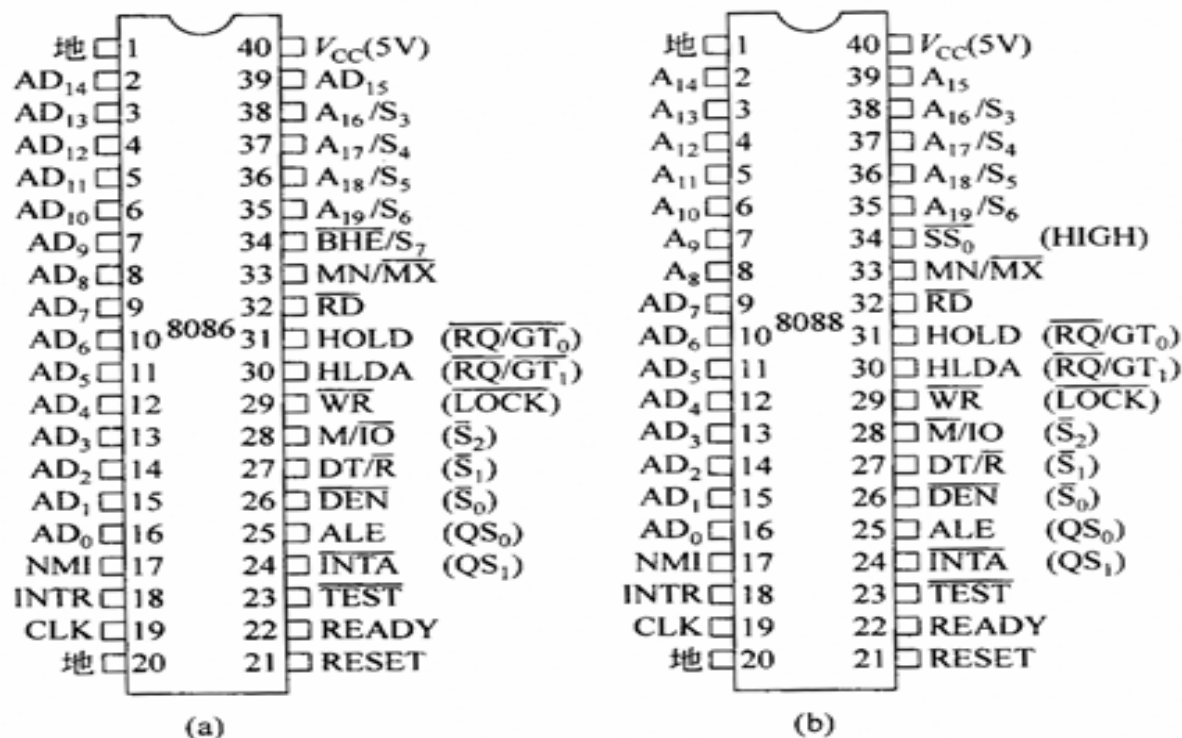


图 2.9 8086/8088 的引脚信号 (括号中为最大模式时引脚名)

2.8086最大模式下引脚的功能定义

- 当 $\overline{\text{MN}}/\overline{\text{MX}}$ 为**低电平**时，8086 CPU工作在最大模式。
- 在最大模式下，许多总线控制信号不是由8086直接产生，而是通过**总线控制器8288**产生的。
- 因此，8086在最小模式下提供的总线控制信号的引脚(24 ~ 31脚)就需重新定义，改作支持最大模式之用。

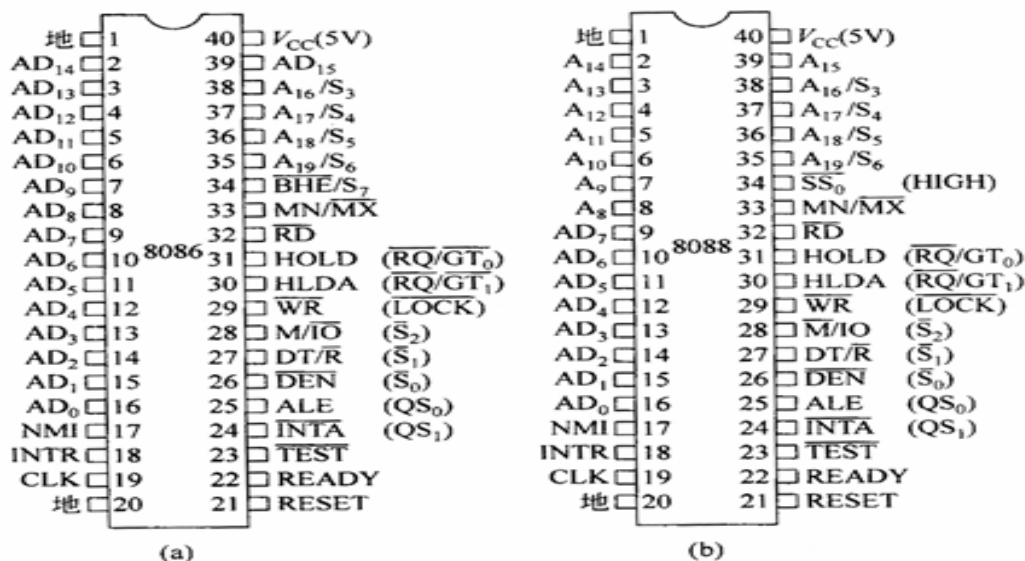
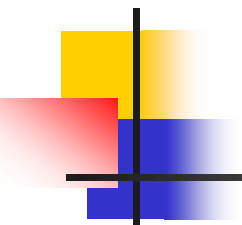


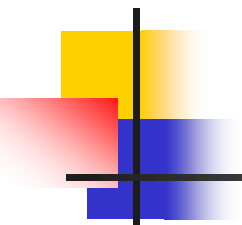
图 2.9 8086/8088 的引脚信号 (括号中为最大模式时引脚名)


$$\overline{S}_2, \overline{S}_1, \overline{S}_0$$

- 总线周期状态信号，输出，三态。
- 8288总线控制器依据这三个状态信号产生访问存储器和I/O端口的控制命令。

表 2.5 $\overline{S}_2 \sim \overline{S}_0$ 对应的总线周期类型

$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	总线周期
0	0	0	INTA 周期
0	0	1	I/O 读周期
0	1	0	I/O 写周期
0	1	1	暂停
1	0	0	取指令周期
1	0	1	读存储器周期
1	1	0	写存储器周期
1	1	1	无源状态



QS₁、QS₀

- 指令队列状态信号，输出。
- 这两个信号组合起来提供了本总线周期的前一个时钟周期中指令队列的状态，以便于外部器件（如8087协处理器）对8086/8088内部指令队列的动作进行跟踪。

表 2.6 QS₁ 和 QS₀ 的代码组合与队列状态

QS ₁	QS ₀	队 列 状 态
0	0	无操作
0	1	从指令队列中取出指令的第 1 个字节
1	0	队列为空
1	1	从指令队列中取出第 1 字节以后部分

$\overline{RQ/GT_1}$ 、 $\overline{RQ/GT_0}$

- 总线请求信号输入/总线请求允许信号输出，双向，低电平有效，三态。
- 供协处理器等发出使用总线的请求信号和接收CPU对总线请求的回答信号。
- 后者比前者的优先级高。

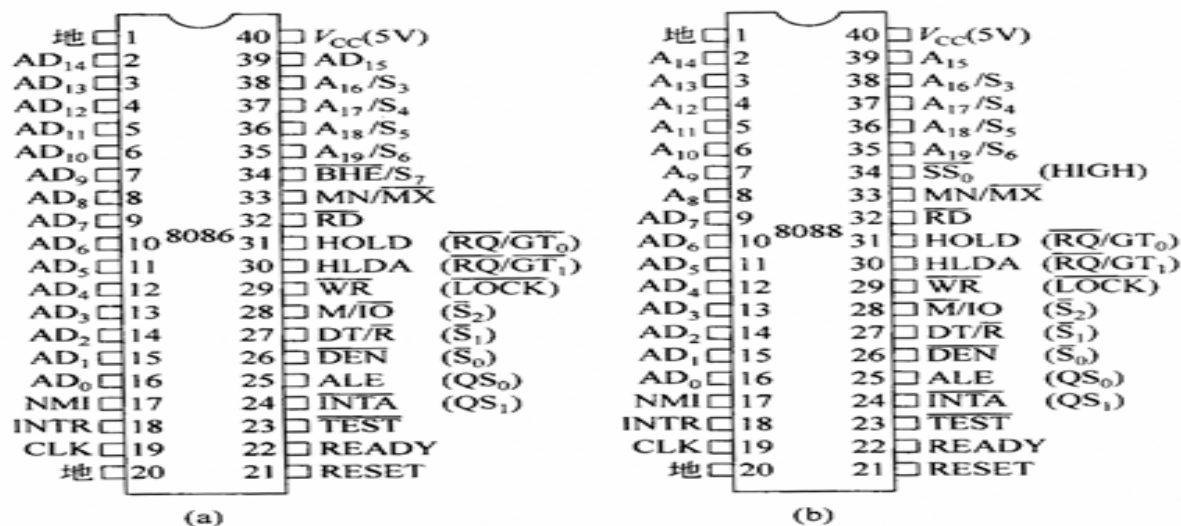
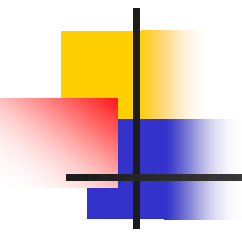


图 2.9 8086/8088 的引脚信号(括号中为最大模式时引脚名)



$\overline{\text{LOCK}}$

- 总线封锁信号，输出，低电平有效，三态。
- 当 $\overline{\text{LOCK}}$ 为低电平时，表示CPU独占总线使用权。
- $\overline{\text{LOCK}}$ 信号由指令前缀LOCK产生，而在LOCK后面的一条指令执行完后，便撤销了 $\overline{\text{LOCK}}$ 信号。
- 此信号是为避免多个处理器使用共有资源时产生冲突而设置的。

3.8088的引脚特性

8088和8086之间引脚上的不同主要表现在：

- (1)由于8088 CPU外部一次只传送8位数据，因此其引脚 $A_{15} \sim A_8$ 仅用于输出地址信号，而8086则将此8条线变为双向分时复用的 $AD_{15} \sim AD_8$ 。
- (2)8086 CPU上的 $\overline{BHE}/S_7$ 信号在8088上变为 $\overline{SS}_0$ (HIGH)信号。这是一条状态输出线，它与 $M/\overline{IO}$ 和 $DT/\overline{R}$ 信号一起，决定了8088 CPU在最小模式下现行总线周期的状态。HIGH在最大模式时始终为高电平输出。
- (3)8088的存储器/外设控制引脚是 $\overline{M}/IO$

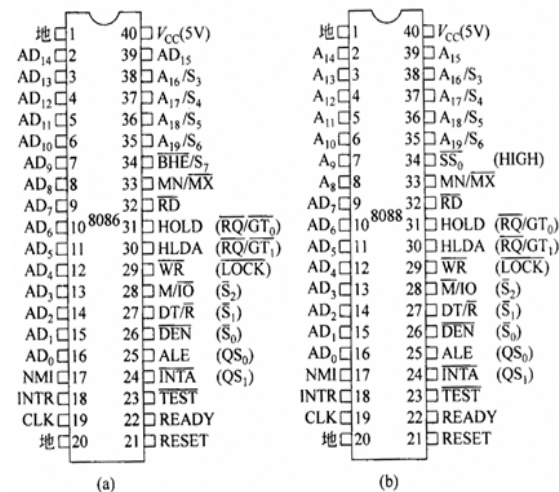
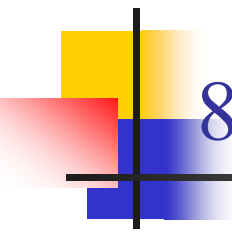
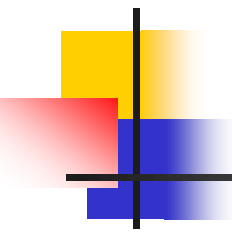


图2.9 8086/8088的引脚信号(括号中为最大模式时引脚名)



8088 CPU在最小模式下总线周期状态

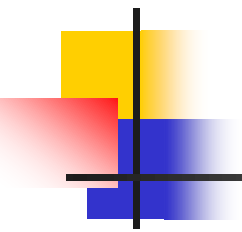
IO/ $\bar{M}$	DT/ $\bar{R}$	$\overline{SSO}$	功能
0	0	0	取操作码
0	0	1	读存储器
0	1	0	写存储器
0	1	1	无效状态
1	0	0	中断响应
1	0	1	读I/O
1	1	0	写I/O
1	1	1	暂停



2.1.4 8086/8088系统的工作模式

包括：

- 最小模式组成
- 最大模式组成



1.最小模式组成

- ▶ 当 $MN/\overline{MX} = 1$ 时，8086 CPU工作在最小模式之下，即单处理器系统方式。
- ▶ 此时，构成的微型机中只包括一个8086或8088 CPU，且系统总线的所有控制信号都由CPU直接给出，系统中的总线控制逻辑电路被减到最少，适合于较小规模的应用。

图2.10 8086最小模式下的典型配置

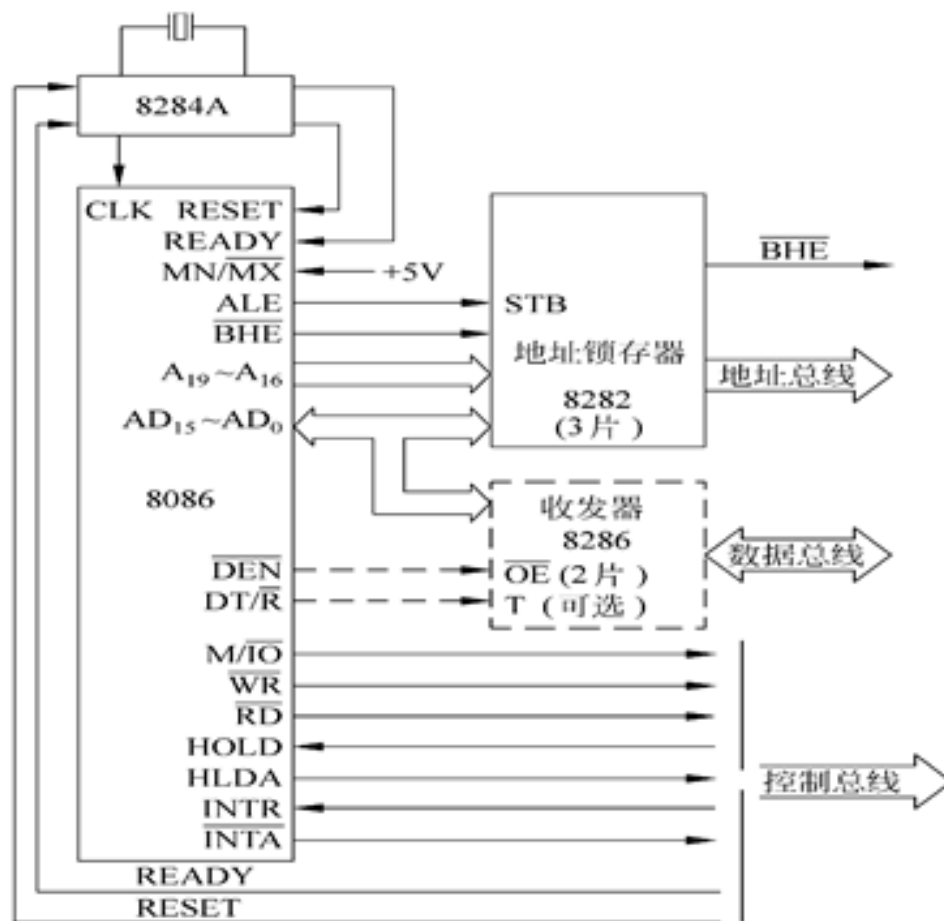
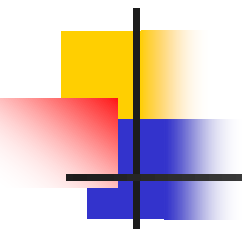


图 2.10 8086 最小模式下的典型配置



1)时钟发生器8284A

- 8086/8088CPU所需的时钟信号由外部的时钟发生器提供。
- 8284A是Intel公司专为8086设计的时钟信号发生器，能产生8086所需的5MHz系统时钟信号，即系统主频。
- 8284A除提供恒定的时钟信号外，还对外界输入的准备就绪信号RDY和复位信号进行同步操作。

图2.11 8284A与8086的连接图

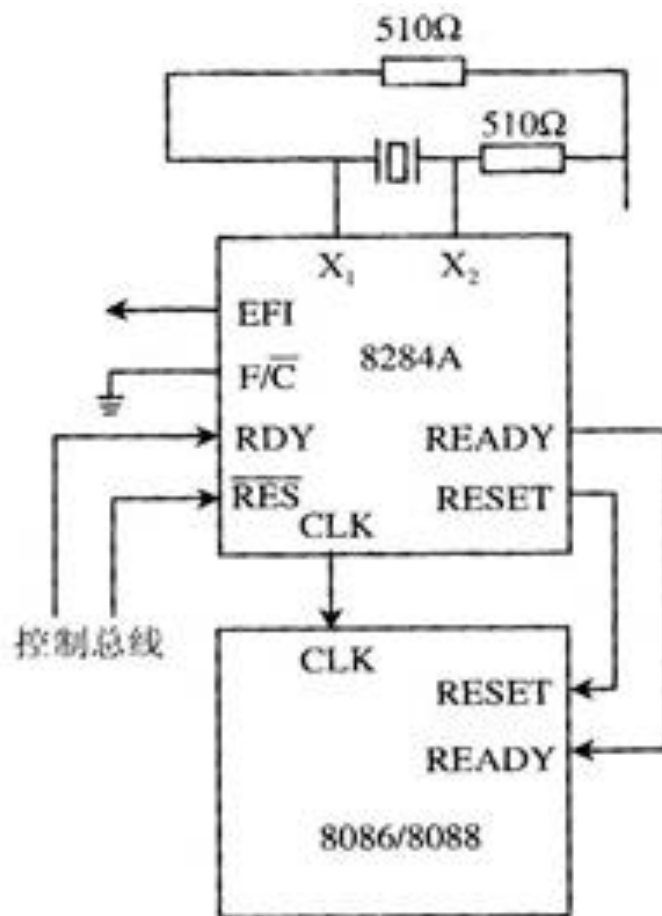
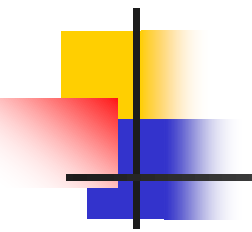


图2.11 8284A 与 8086 的连接



2)地址锁存器8282

- 由于8086/8088的部分地址线和数据线采用**分时复用**的技术，在一个总线周期内总线**首先传送地址，然后传送数据**。
- 在每个总线周期的 T_1 状态利用地址锁存允许信号ALE的后沿，将地址信息锁存到地址锁存器内，**经锁存后的地址信号可以在整个总线周期保持不变，从而为外部提供稳定的地址信息**。
- Intel 8282是具有三态缓冲的单向8位锁存器。使用时，将8282的选通信号输入端STB与ALE相连。
- 当ALE有效时，8086的地址信号被锁存并以同相方式传至输出端，供存储器芯片和I/O接口芯片使用。
- 8086除了20位地址外， $\overline{\text{BHE}}$ 也要锁存，所以**需要3片8282**作为锁存器。

图2.12 8282锁存器与8086的连接

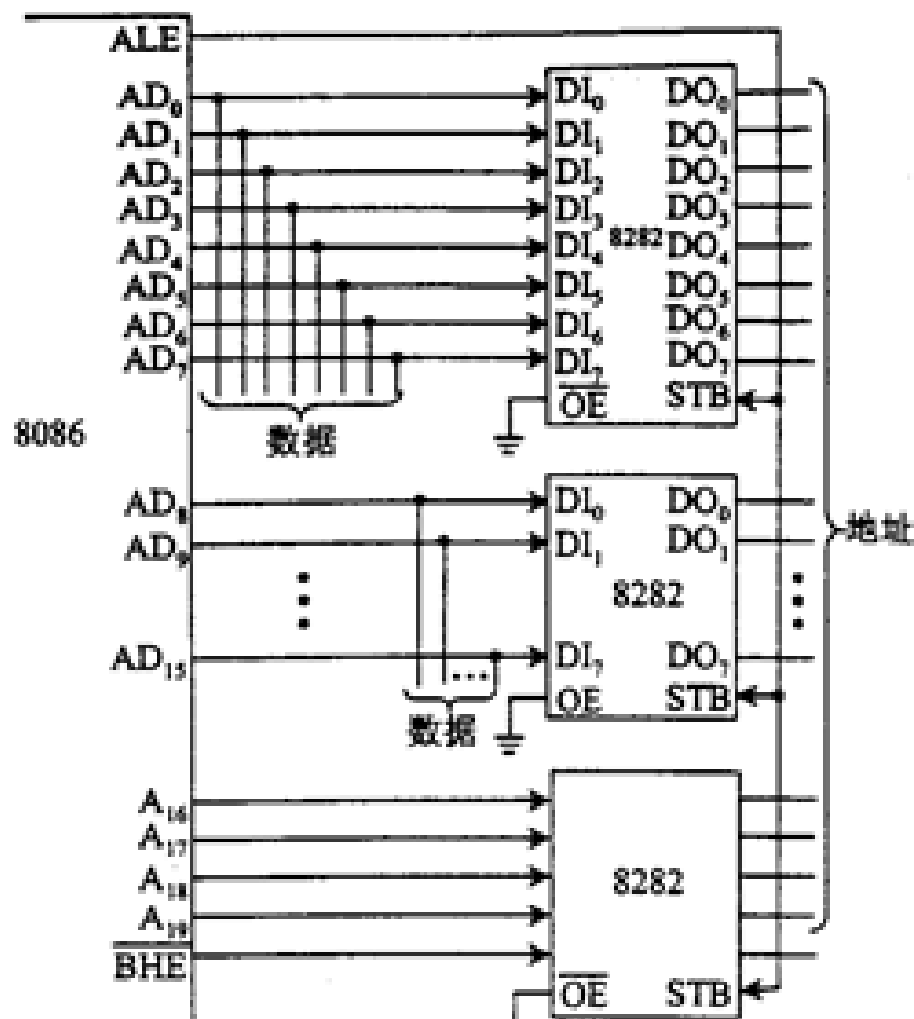
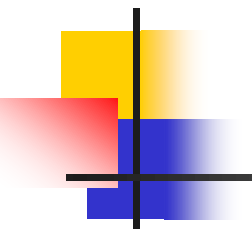


图 2.12 8282 锁存器与 8086 的连接



3)数据收发器8286

- 当系统中所连接的存储器及I/O设备较多时，为了使系统能稳定工作，可以采用发送器和接收器来增加驱动能力。发送器和接收器简称为**收发器**，也常称为**总线驱动器**。
- Intel 8286是8位双向三态缓冲器。8286的数据线有两组， $A_7 \sim A_0$ 、 $B_7 \sim B_0$ ，引脚T用来控制数据传输的方向，当 $T=1$ 时，方向为 $A \rightarrow B$ ；当 $T=0$ 时，方向为 $B \rightarrow A$ 。OE 是输出允许信号，当它为0时，允许数据传送。
- 8086的数据总线是16位，如果要选用8286做总线驱动器，则需要2片。
- 如果是较小规模的最小模式系统，不需要总线驱动器，那么就用CPU的 $\overline{M/IO}$ 、 $\overline{RD}$ 、 $\overline{WR}$ 组合起来决定系统中数据传输的方式。

图2.13 8286收发器和8088的连接

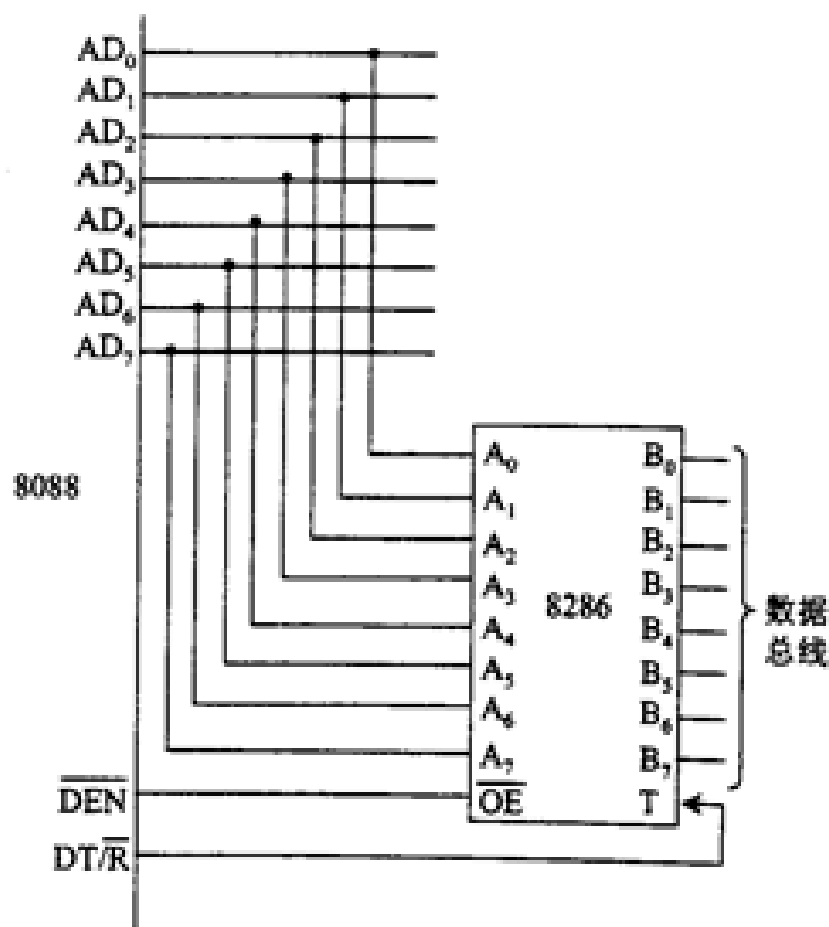
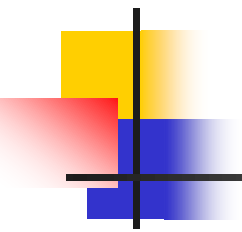


图 2.13 8286 收发器和 8088 的连接



2.最大模式组成

- 当 $\overline{MN}/\overline{MX} = 0$ 时，8086 CPU工作在最大模式。
- 在最大模式下，构成的微型计算机中除了有8086 CPU之外，还可以接另外的CPU，如8087、8089等，以构成**多处理器系统**。
- 在最大模式系统中，系统的许多控制信号不再由8086直接发出，而是由**总线控制器8288**对8086发出的控制信号进行变换和组合，从而得到各种系统控制信号。

1)8086最大模式典型配置

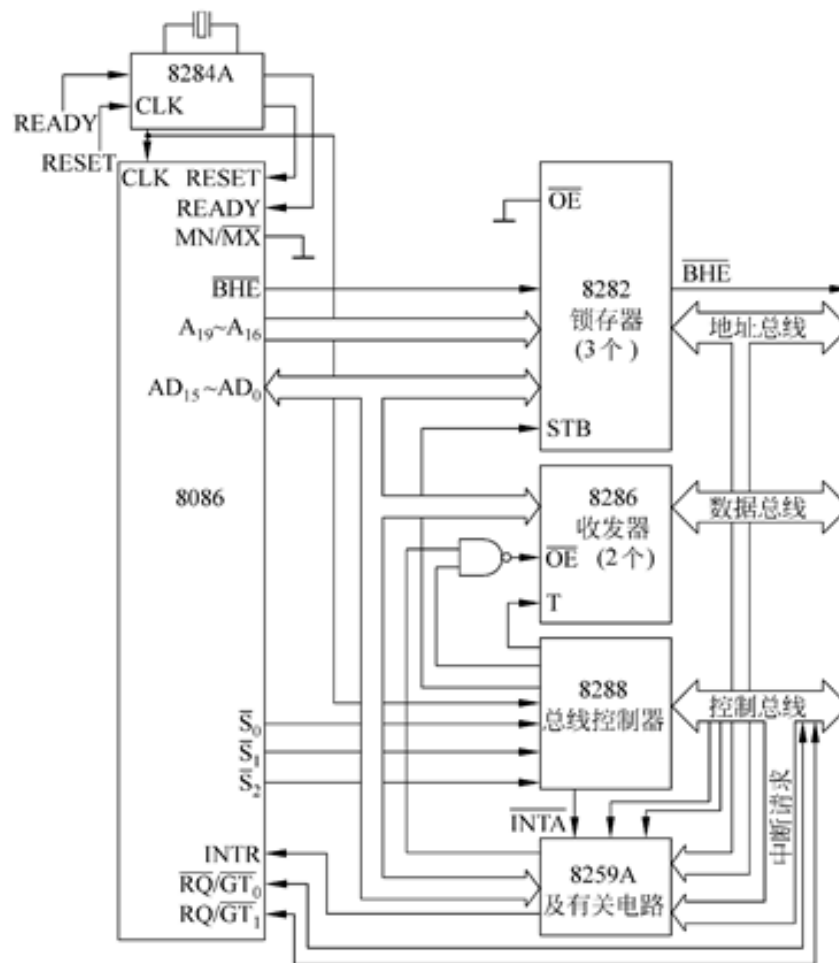


图 2.14 8086 最大模式典型配置

2)8288总线控制器主要引脚信号

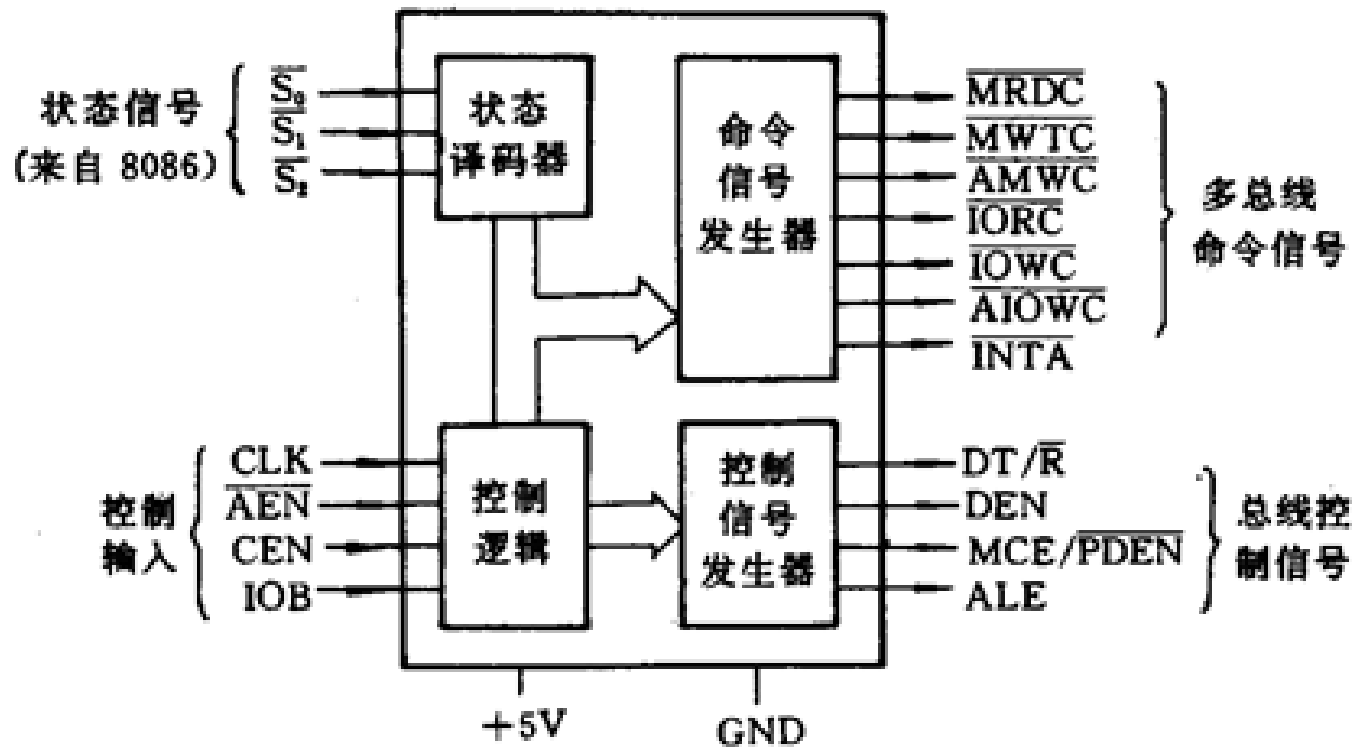
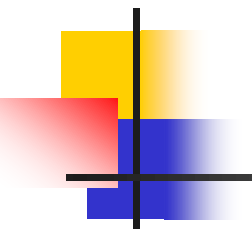


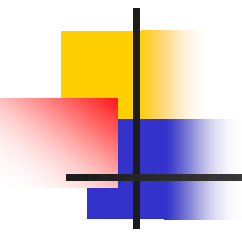
图 2.15 8288 的原理框图



2.1.6 8086/8088的操作和时序

本小节内容：

- 系统的复位和启动操作
- 总线读/写操作



1.系统的复位和启动操作

- RESET引脚至少维持4个时钟周期的高电平信号时，8086/8088复位。如果是初次加电引起的复位，则要求维持不小于50 μ s的高电平。
- 8086/8088复位后将从内存的FFFF0H处开始执行指令。因此，一般在该处放一条无条件转移指令，转移到系统程序的入口处。这样系统一旦被启动，便自动进入系统程序。

图2.16 8086的复位操作时序

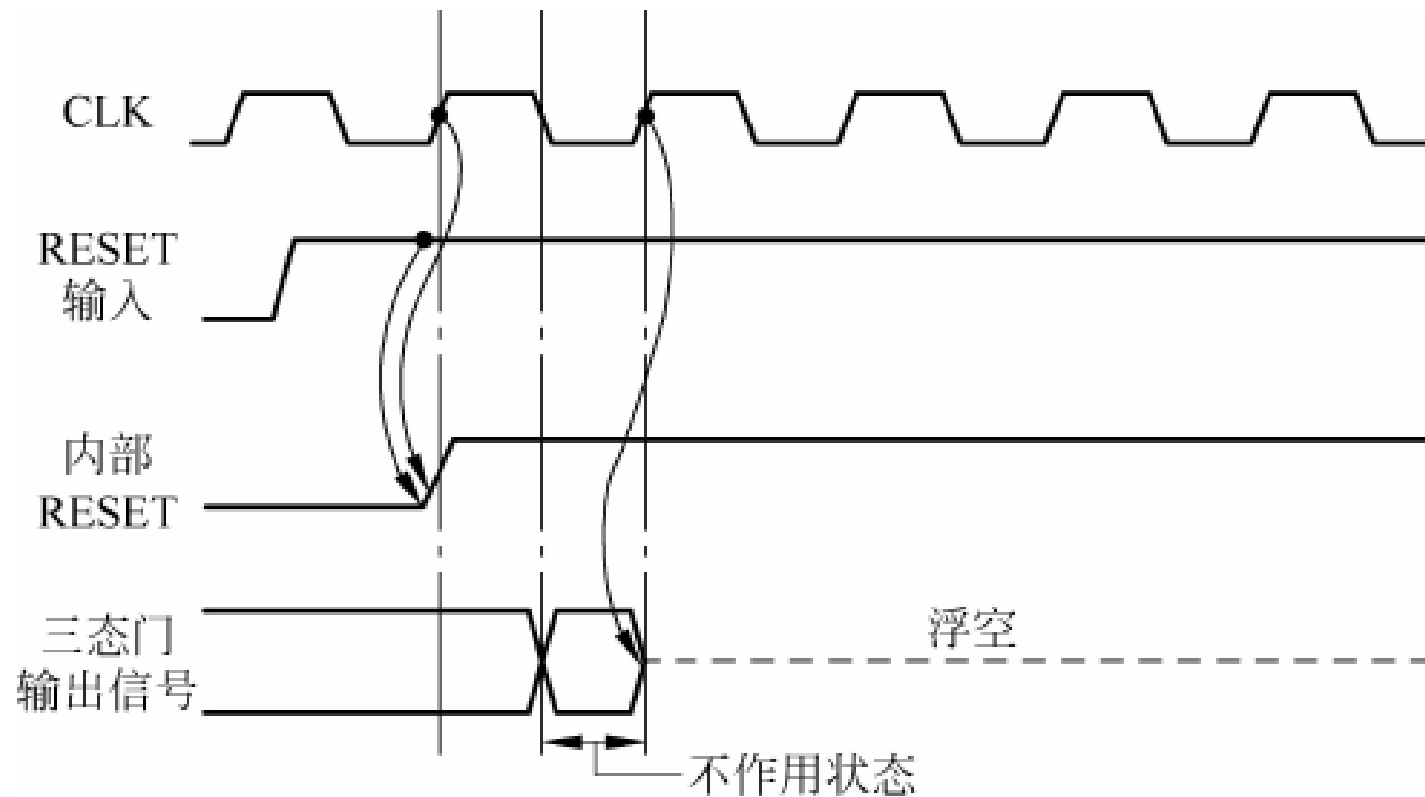
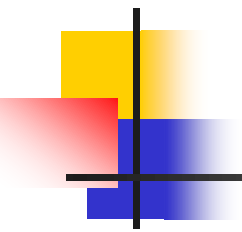


图 2.16 8086 的复位时序



2.总线读/写操作

可以分为：

- 总线读操作(CPU从存储器或I/O端口读取数据)
- 总线写操作(CPU将数据写入存储器或I/O端口)

(1)最小模式下总线读周期

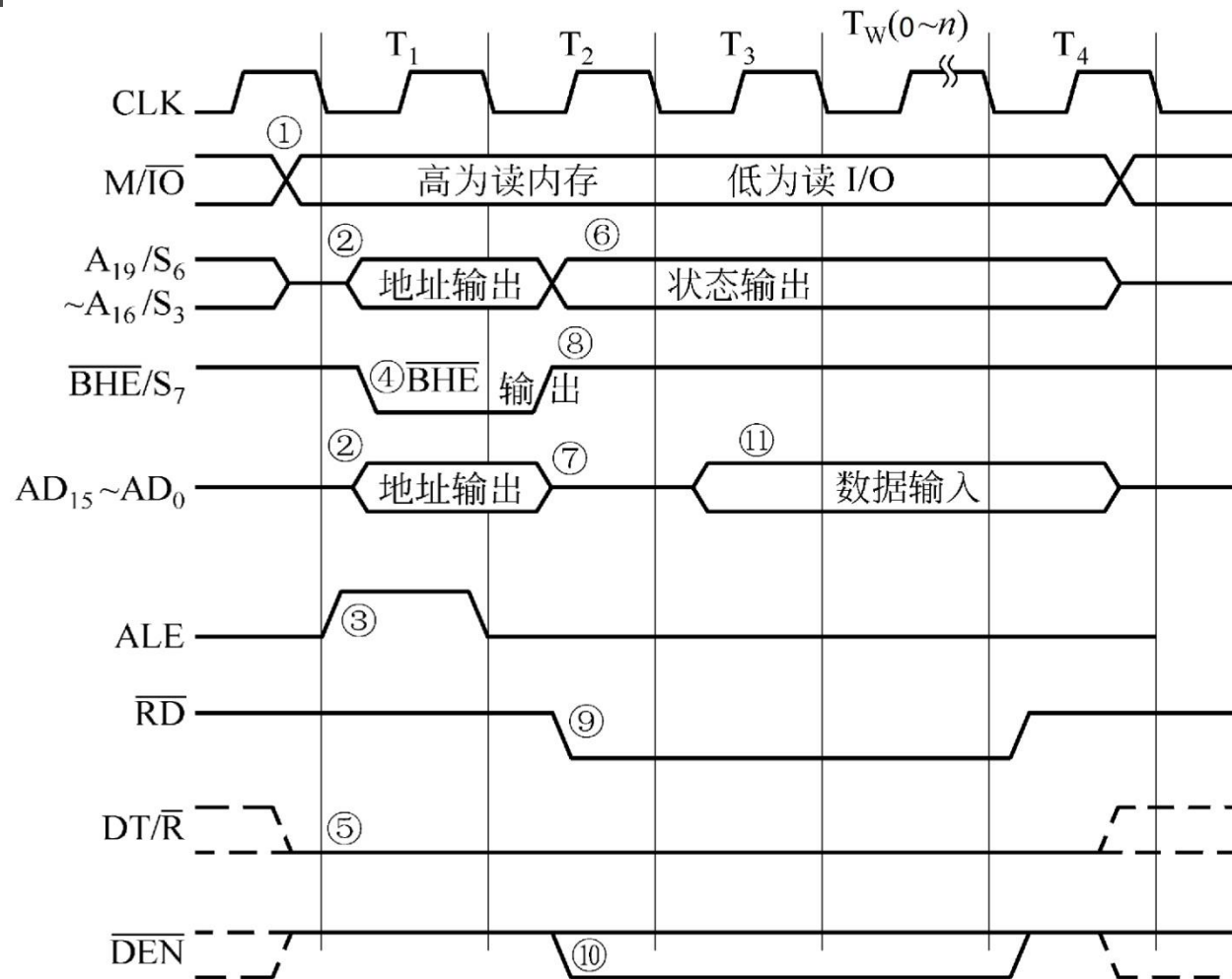


图2.17 8086/8088 CPU最小模式下总线读周期时序

(2)最小模式下总线写周期

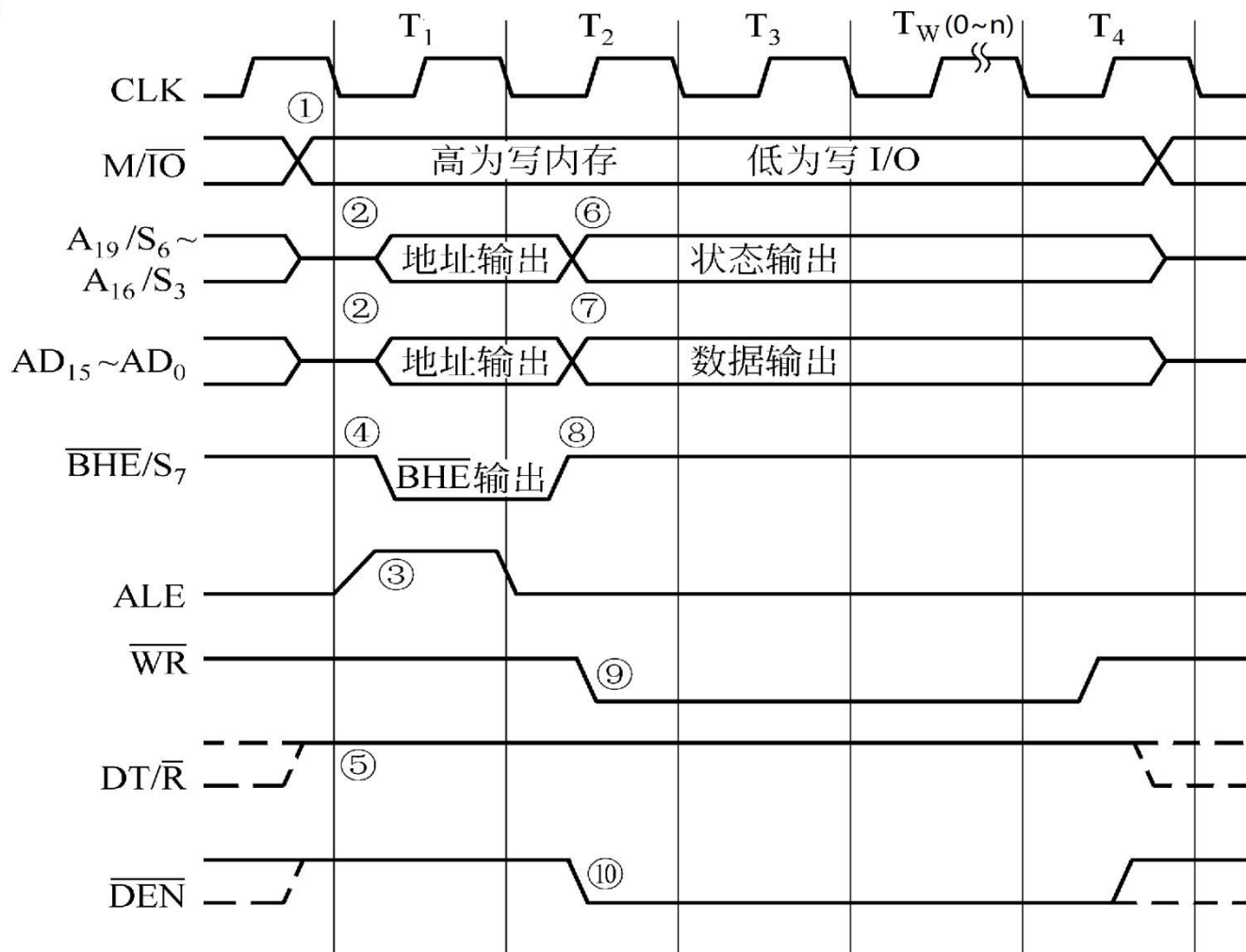


图2.18 8086/8088 CPU最小模式下总线写周期时序

(3)最大模式下的总线读操作

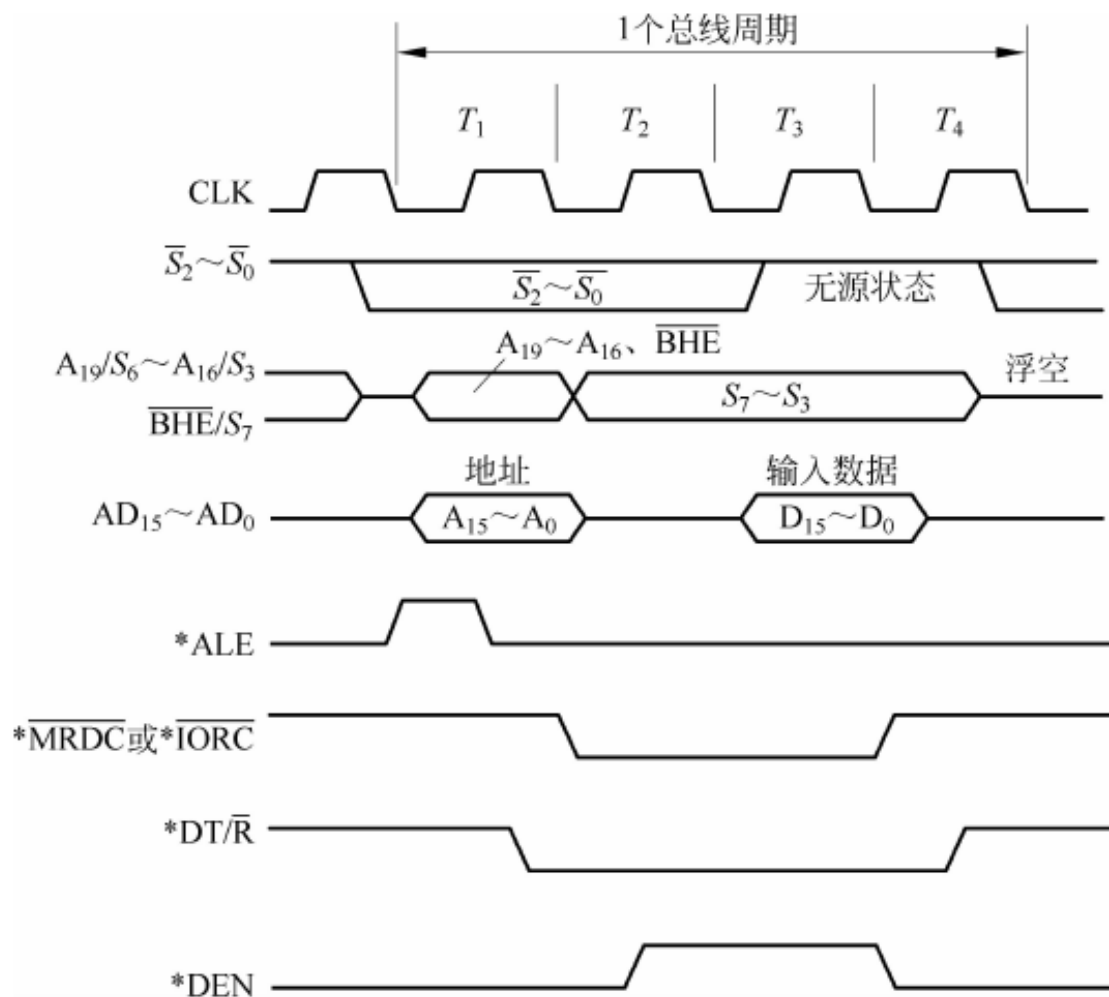


图 2.19 最大模式下的总线读操作时序

(4)最大模式下的总线写操作

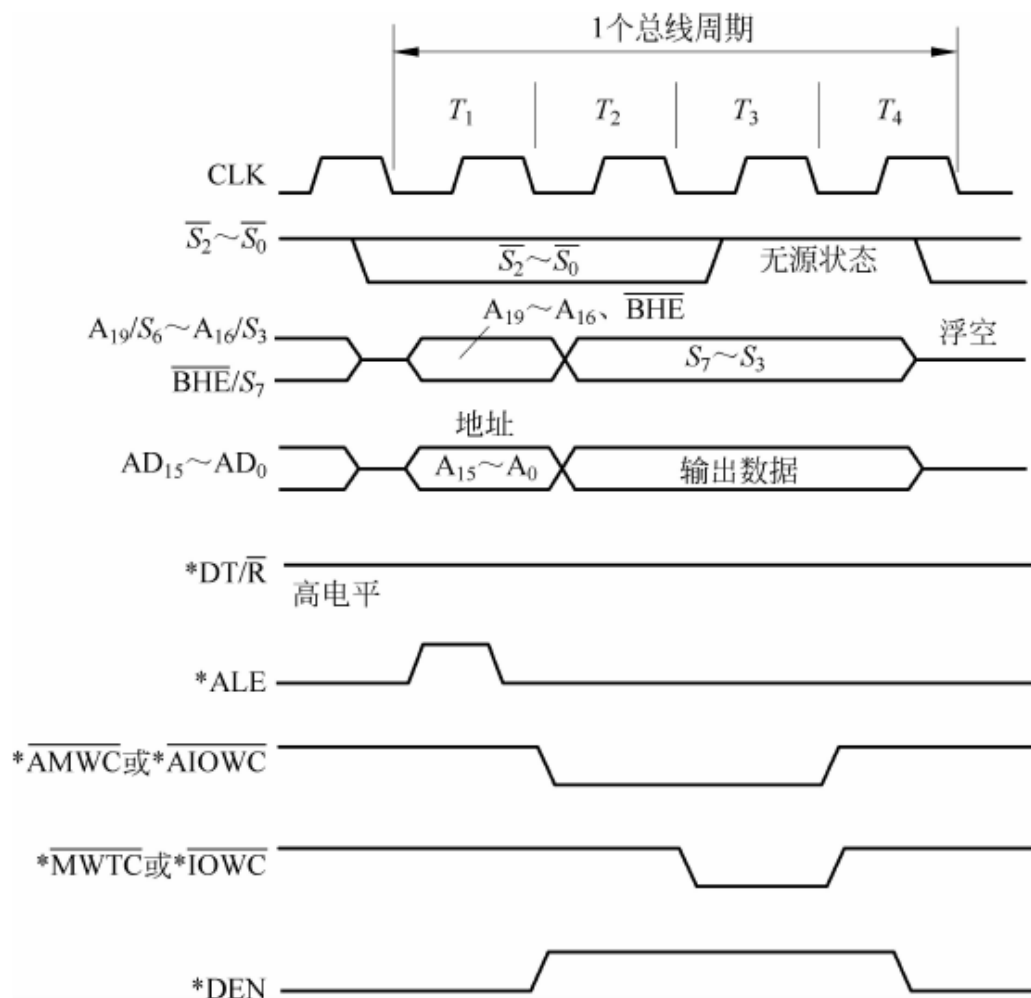


图 2.20 最大模式下的总线写操作时序

3.中断响应操作

当8086/8088CPU收到外界从INTR引脚上送来的中断请求信号，并且满足 $IF=1$ 时，CPU在执行完当前指令后，便执行一个中断响应时序。

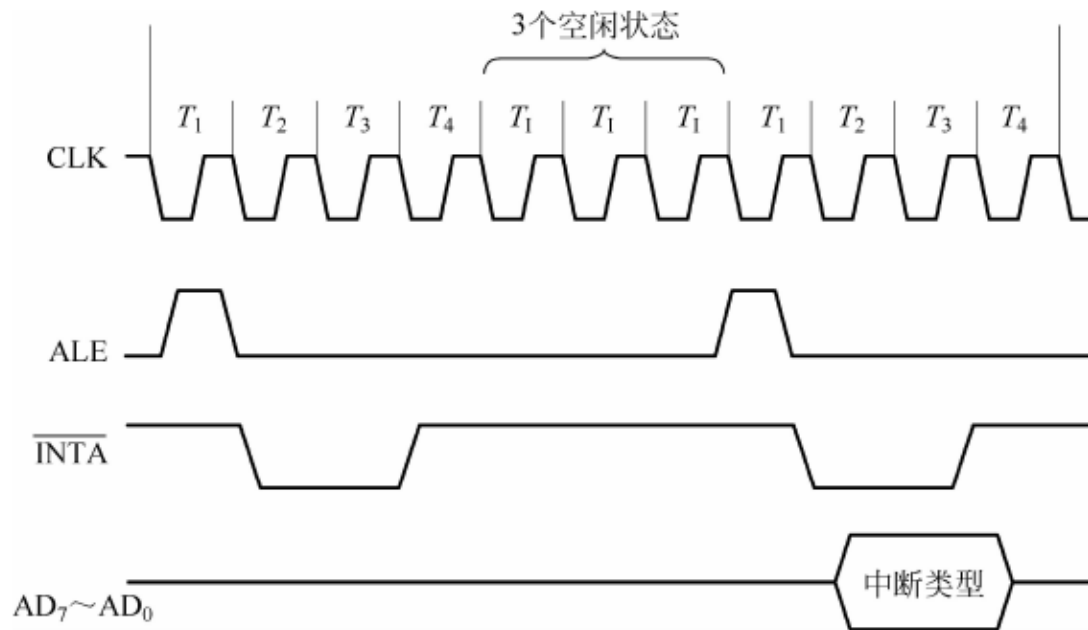


图 2.21 8086 中断响应的时序